

NoSQL na prática: do documento ao índice invertido

*Uma aula construída sobre dois laboratórios —
MongoDB como sistema de registro, Elastic Stack como camada de busca.*

DISCIPLINA	Bancos de Dados Não Relacionais
PROFESSOR	Raul Carvalho de Souza
ALUNO	Guilherme Levi
DURAÇÃO	300 minutos · 8 módulos
BASE	Lab 1 — MongoDB 8.2 · Lab 2 — Elasticsearch 8.15, Kibana e Logstash

Todo comando e toda saída deste material foram executados de verdade nos dois laboratórios. Inclui 10 exercícios, 8 armadilhas reais de ambiente e 10 questões de avaliação.

Sumário

Mód. 1	Por que NoSQL: o problema antes da solução	25 min
Mód. 2	MongoDB: o modelo de documento e o CRUD	45 min
Mód. 3	MongoDB: o Aggregation Framework	40 min
Mód. 4	MongoDB: indexação e desempenho	35 min
Mód. 5	Elasticsearch: arquitetura e o índice invertido	45 min
Mód. 6	Elasticsearch: mapping, ingestão e consulta	40 min
Mód. 7	Busca vetorial, busca híbrida e RAG	40 min
Mód. 8	Os dois juntos: sistema de registro e camada de busca	30 min

MATERIAL COMPLEMENTAR

A	O que o roteiro não conta: armadilhas reais	8 casos
B	Exercícios com solução comentada	10 exercícios
C	Avaliação, tabela de decisão e glossário	10 questões

Por que NoSQL: o problema antes da solução

O modelo relacional resolve bem um conjunto de problemas há cinquenta anos. A pergunta desta aula não é o que há de errado com ele — é que problemas apareceram depois que ele já estava consolidado, e por que exigiram estruturas diferentes.

OBJETIVOS DE APRENDIZAGEM

- Identificar as três pressões concretas que motivaram os bancos não relacionais: esquema móvel, escala horizontal e busca por relevância.
- Situar MongoDB e Elasticsearch dentro da taxonomia NoSQL, sabendo dizer a que família cada um pertence.
- Enunciar o teorema CAP com precisão, incluindo o que a versão popular dele distorce.
- Sustentar a tese central da aula: os dois sistemas dos laboratórios não competem, se complementam.

CONCEITOS

NoSQL não quer dizer "sem SQL"

O termo pegou como "Not Only SQL". Ele não descreve uma tecnologia, descreve uma recusa: a de tratar tabela com esquema fixo como a única forma de organizar dados. O que une sistemas tão diferentes quanto Redis, Cassandra, Neo4j, MongoDB e Elasticsearch não é uma característica comum positiva — é o fato de nenhum deles ser um banco relacional. Por isso a pergunta "NoSQL é melhor que SQL?" não tem resposta: é como perguntar se um veículo terrestre é melhor que um Fusca.

ERRO COMUM

Tratar NoSQL como categoria única e concluir que "NoSQL não tem transação" ou "NoSQL não tem esquema". O MongoDB tem transação multi-documento desde a versão 4.0 e permite validação de esquema por coleção. A generalização é falsa para praticamente qualquer afirmação.

A taxonomia: cinco famílias, dois representantes

Chave-valor (Redis, DynamoDB) guarda um valor opaco sob uma chave e não sabe nada sobre o conteúdo. Documento (MongoDB, CouchDB) guarda estruturas aninhadas e sabe consultar campos dentro delas. Coluna larga (Cassandra, HBase) organiza por família de colunas e otimiza escrita em volume. Grafo (Neo4j) trata a relação como cidadã de primeira classe. Motor de busca (Elasticsearch, Solr) inverte o problema: em vez de ir do documento ao conteúdo, vai do termo aos documentos. Os dois laboratórios cobriram justamente as duas famílias mais usadas juntas.

ANALOGIA

Chave-valor é o guarda-volumes: você entrega uma mala, recebe uma ficha, e ninguém olha dentro. Documento é o arquivo de pastas: cada pasta tem estrutura própria e dá para procurar por um campo específico. Motor de busca é o índice remissivo no fim do livro: ele não guarda o texto, guarda em que páginas cada palavra aparece.

ACID e BASE

ACID (atomicidade, consistência, isolamento, durabilidade) é a garantia de que uma transação ou acontece inteira ou não acontece. BASE (basically available, soft state, eventually consistent) troca parte dessa garantia por disponibilidade e escala: aceita-se que réplicas divirjam por um intervalo, desde que converjam depois. A escolha não é ideológica, é de domínio: saldo bancário exige ACID; contador de visualizações de vídeo tolera BASE com folga.

ERRO COMUM

Assumir que "NoSQL é BASE". O MongoDB com write concern majority e read concern majority oferece garantias fortes; o Elasticsearch, por outro lado, é near real-time por projeto e nunca deve ser tratado como fonte da verdade transacional.

Teorema CAP, com honestidade

O enunciado correto é: quando ocorre uma partição de rede (P), o sistema precisa escolher entre continuar respondendo com dado possivelmente desatualizado (A) ou recusar a resposta para não violar consistência (C). Não é "escolha dois dos três" no dia a dia: partição de rede não é uma opção de projeto, é um evento que acontece. Fora de partição, um sistema pode perfeitamente ser consistente e disponível ao mesmo tempo. A formulação PACELC é mais útil: na partição, escolha entre A e C; caso contrário (Else), escolha entre latência (L) e consistência (C).

ERRO COMUM

Dizer que "MongoDB é CP e Cassandra é AP" como se fosse propriedade fixa. Em ambos, o comportamento depende de configuração — write concern, read preference, nível de consistência por consulta. A classificação é de configuração, não de produto.

A tese desta aula

Os dois laboratórios não foram exercícios independentes. O Lab 2 montou, sem anunciar, uma arquitetura de produção clássica: o MongoDB do Lab 1 emitindo logs, o Logstash recebendo, o Elasticsearch indexando, o Kibana exibindo. Essa é a divisão de trabalho real: um sistema guarda a verdade com garantia de escrita; o outro constrói um índice derivado, otimizado para uma pergunta que o primeiro responde mal. Toda a aula vai voltar a esse ponto.

ANALOGIA

Uma biblioteca tem o acervo e tem o catálogo. O acervo é a verdade: se o catálogo sumir, reconstrói-se a partir dos livros. Se o acervo sumir, o catálogo vira uma lista de coisas que não existem mais. O Elasticsearch é o catálogo.

NA PRÁTICA

A arquitetura que o laboratório montou sem dizer

```
$ docker ps --format "table {{.Names}}\t{{.Status}}"
```

SAÍDA REAL

NAMES	STATUS
mongodb-container	Up 11 hours
kibana	Up 11 hours
logstash	Up 11 hours
elasticsearch	Up 11 hours (healthy)

Quatro contêineres, dois bancos, dois papéis. O MongoDB é o sistema de registro. Os outros três formam a camada de busca e observabilidade. Nenhum documento de negócio é escrito diretamente no Elasticsearch: tudo que chega lá foi derivado de outra fonte.

Ambiente dos Labs 1 e 2, verificado em 09/09/2026

O elo entre os dois laboratórios está em três linhas de configuração

```
# lab01-mongodb/docker-compose.yml
logging:
  driver: gelf
  options:
    gelf-address: "udp://localhost:12201"
    tag: "mongodb-log"
```

SAÍDA REAL

health	status	index	docs.count
yellow	open	mongodb-logs-2026.09.09	86

O driver de log GELF do Docker desvia a saída do MongoDB para a porta 12201/UDP, onde o Logstash escuta. O MongoDB não sabe que o Elasticsearch existe, e o Elasticsearch não sabe o que é um MongoDB. O acoplamento é de um lado só, e por isso o sistema tolera a queda da camada de busca.

lab01-mongodb/docker-compose.yml e logstash/pipeline/logstash.conf

FECHAMENTO

Guarde a divisão de papéis: um sistema responde "qual é o estado atual deste registro?" com garantia; o outro responde "que documentos são relevantes para esta pergunta?" com velocidade. Os próximos três módulos tratam do primeiro. Depois, os três seguintes tratam do segundo.

PARA DISCUTIR EM SALA

1. Se o MongoDB do laboratório sabe emitir log e o Elasticsearch sabe indexá-lo, por que não gravar o log direto no Elasticsearch e eliminar o Logstash?
2. Um sistema de e-commerce precisa de catálogo com busca por texto e de carrinho com consistência forte. Que arranjo você proporia, e onde ficaria a fonte da verdade?
3. Em que situação concreta você aceitaria que dois usuários vissem valores diferentes do mesmo dado por alguns segundos?

MongoDB: o modelo de documento e o CRUD

Um documento não é uma linha com JSON dentro. A diferença muda como se modela, como se consulta e onde se erra — e o primeiro laboratório produziu vários desses erros de forma instrutiva.

OBJETIVOS DE APRENDIZAGEM

- Explicar a hierarquia database → coleção → documento e por que a criação é preguiçosa.
- Distinguir BSON de JSON e prever o efeito prático da tipagem numa consulta.
- Usar os operadores de atualização corretamente, sabendo o que acontece quando se omite o \$set.
- Escolher entre deleteOne e deleteMany com consciência do risco de cada um.
- Aplicar o princípio do menor privilégio com roles de banco.

CONCEITOS

Criação preguiçosa de banco e coleção

O comando use não escreve nada em disco: ele apenas aponta a variável db para um nome. O banco passa a existir no momento em que o primeiro documento é gravado, e o mesmo vale para a coleção. Por isso o createCollection é opcional — ele só é necessário quando se precisa declarar alguma opção no ato da criação, como validação de esquema, coleção capped (tamanho fixo, sobrescrevendo os documentos mais antigos) ou time-series. Fora desses casos, ele serve apenas para tornar a intenção explícita no código.

ERRO COMUM

Rodar um update contra uma coleção cujo nome foi digitado errado. O MongoDB não reclama: ele cria a coleção vazia e reporta zero documentos modificados. O erro passa despercebido porque não há mensagem de erro — foi exatamente o que aconteceu na Parte 7 do Lab 1, cujo roteiro mandava usar db.estudantes numa altura em que a coleção ainda se chamava alunos.

BSON: a tipagem que o JSON não tem

O MongoDB armazena BSON, um formato binário que estende o JSON com tipos que o JSON não possui: inteiro de 32 e 64 bits, double, decimal128, data, binário, ObjectId. Isso não é detalhe de implementação — é o que permite comparar datas e números corretamente. A string "60" e o número 60 são valores distintos e uma busca por um não encontra o outro. Pior: o operador \$gt aplicado a strings compara em ordem alfabética, de modo que "9" é maior que "10".

ERRO COMUM

Importar dados de CSV sem converter tipos e depois estranhar que { idade: { \$gt: 20 } } não retorne ninguém, porque todas as idades foram gravadas como string.

ObjectId: doze bytes que carregam informação

Quando não se fornece um `_id`, o driver gera um ObjectId de 12 bytes: 4 bytes de timestamp Unix, 5 bytes de valor aleatório por processo e 3 bytes de contador incremental. Duas consequências práticas: ordenar por `_id` ordena aproximadamente por ordem de inserção, e o identificador é gerado no cliente, o que permite inserir sem ida ao servidor para reservar um número — diferente de uma sequência de banco relacional.

ANALOGIA

É uma etiqueta de encomenda que já traz a data e a agência de origem impressas. Você não precisa consultar a central para saber quando foi postada.

Cursor: find devolve um ponteiro, não os dados

O `find()` devolve um cursor — uma referência que o servidor vai entregando em lotes, para não carregar toda a coleção na memória de uma vez. É por isso que ele aceita encadear `.sort()`, `.limit()` e `.skip()`: esses métodos alteram a consulta antes de ela ser executada, não filtram um resultado já pronto. O `findOne()` devolve o documento em si e por isso não aceita encadeamento algum.

ERRO COMUM

Usar `.skip()` com valores altos para pagnar. O servidor precisa percorrer e descartar todos os documentos pulados, então a página 1000 custa muito mais que a página 1. Paginação por faixa de valor (range pagination) escala; skip não.

O \$set é obrigatório — e omiti-lo destrói o documento

Os operadores de atualização dizem o que fazer com o documento encontrado. O `$set` cria o campo se não existir e sobrescreve se existir; o `$unset` remove a chave; o `$inc` soma; o `$push` acrescenta a um array. Se o segundo argumento do `updateOne` não usar operador nenhum, o MongoDB interpreta como substituição integral: o documento inteiro vira o objeto passado, e todo campo que não estava ali é perdido.

ERRO COMUM

Escrever `updateOne({nome:"Ana Costa"}, {idade:24})` esperando alterar só a idade. O resultado é um documento que contém apenas idade: 24 — nome, curso e todo o resto foram apagados, sem aviso.

Remover a chave não é gravar null

Depois de um `$unset`, o campo deixa de existir no documento e passa a ser encontrado por `{ campo: { $exists: false } }`. Se em vez disso fosse gravado null, o campo continuaria existindo com valor nulo, e a consulta por `$exists: false` não o encontraria. A distinção importa em agregação: campos ausentes são ignorados pelo `$avg`, enquanto null participa da conta.

ANALOGIA

Apagar a linha do formulário é diferente de escrever "não se aplica" nela. Nos dois casos não há valor útil, mas só no segundo há registro de que a pergunta foi feita.

matchedCount e modifiedCount são números diferentes

O retorno de um update informa quantos documentos casaram com o filtro (matchedCount) e quantos efetivamente mudaram (modifiedCount). Eles divergem quando o valor novo é igual ao que já estava gravado: o documento casou, mas nada foi escrito. Ler os dois números é a forma de saber se a operação fez o que se esperava.

ERRO COMUM

Verificar apenas modifiedCount e concluir que o filtro não encontrou nada, quando na verdade encontrou e o dado já estava correto.

Menor privilégio com roles

O usuário é criado dentro de um banco, e esse banco vira o authSource dele — o lugar onde o servidor procura a credencial ao autenticar. A role read libera find, count e aggregate; a readWrite acrescenta insert, update, delete, createCollection e createIndex. Nenhuma das duas permite criar usuário ou alterar permissão, o que exige roles administrativas. O ponto crucial é que a permissão é verificada pelo servidor a cada operação, não no momento da conexão.

ERRO COMUM

Concluir que a credencial está correta porque a conexão foi aceita. Um usuário com role read conecta normalmente e só descobre o limite quando tenta escrever.

NA PRÁTICA

Tipagem: o que se grava é o que se busca

```
db.disciplinas.insertMany([
  { nome: "Bancos de Dados Não Relacionais", carga_horaria: 60, obrigatoria: true },
  { nome: "Tópicos em Big Data", carga_horaria: 40, obrigatoria: false }
]);
```

carga_horaria é número e obrigatoria é booleano — não são strings. A escolha parece trivial no insert e cobra o preço na consulta: gravado como "60", o campo não é encontrado por { carga_horaria: 60 } nem comparado corretamente por \$gt.

GABARITO.md, Parte 5, EX 5.2

Esquema flexível e o preço dele na agregação

```
db.alunos.insertOne({ nome: "Lucas Moreira", idade: 26,
                     curso: "Data Science", cidade: "Brasília" });

// mais tarde, agrupando por um campo que só um documento tem:
db.estudantes.aggregate([
  { $group: { _id: "$cidade", total: { $sum: 1 } } }
]);
```

SAÍDA REAL

```
[ { _id: null, total: 2 }, { _id: "Brasília", total: 1 } ]
```

O banco aceitou um documento com um campo que nenhum outro possui, sem reclamar. A conta chega no \$group: todo documento sem o campo de agrupamento cai num grupo de _id: null. Isso não é erro, é a regra — mas quem não a conhece lê o resultado como se fosse defeito de dados.

GABARITO.md, Parte 5 (EX 5.3) e Parte 10 (EX 10.3)

A permissão é verificada a cada operação, não na conexão

```
db.createUser({
  user: "leitura_user", pwd: "senha123",
  roles: [ { role: "read", db: "dbNoSQLBD" } ]
});

$ mongosh "mongodb://leitura_user:senha123@localhost:27017/dbNoSQLBD"
> db.estudantes.find()      // funciona
> db.estudantes.insertOne({ nome: "teste" })
```

SAÍDA REAL

```
MongoServerError: not authorized on dbNoSQLBD to execute command insert
```

A conexão foi aceita sem qualquer aviso. O limite só apareceu na primeira escrita. É esse o comportamento que torna o menor privilégio uma defesa real: se a credencial de um serviço de relatório vazar, o atacante herda apenas leitura.

GABARITO.md, Parte 11, EX 11.2

deleteMany não tem desfazer

```
// disciplina de trabalho: rodar o filtro como find antes
db.temp.find({});           // 1. conferir na tela o que vai sumir
db.temp.deleteMany({});     // 2. só então trocar find por deleteMany
```

SAÍDA REAL

```
{ acknowledged: true, deletedCount: 3 }
```

A coleção temp continuou existindo, vazia, com seus índices — deleteMany remove documentos, não a coleção. Quem apaga a coleção do catálogo é o drop(). Fora de transação não há rollback, então conferir o filtro com find antes é a única rede de proteção.

GABARITO.md, Parte 8, EX 8.2 e 8.3

FECHAMENTO

O modelo de documento troca a rigidez do esquema por flexibilidade, e cobra essa troca em disciplina: tipos consistentes, operadores explícitos e conferência antes de apagar. Com o CRUD estabelecido, a próxima pergunta é como extrair informação agregada de uma coleção — que é onde o MongoDB se distancia de vez do modelo relacional.

PARA DISCUTIR EM SALA

1. Por que o MongoDB não avisa quando você atualiza uma coleção que não existe? Que decisão de projeto está por trás disso?
2. Um campo ausente e um campo com null se comportam igual no find? E no \$avg?
3. Se ordenar por `_id` equivale a ordenar por data de inserção, por que ainda assim gravar um campo `criado_em`?
4. Que role você daria a um serviço que gera relatórios noturnos, e por quê?

MongoDB: o Aggregation Framework

O find responde "quais documentos". A agregação responde "o que esses documentos dizem em conjunto". A diferença entre as duas perguntas é a diferença entre consultar e analisar.

OBJETIVOS DE APRENDIZAGEM

- Descrever o pipeline como sequência de estágios e prever o formato da saída de cada um.
- Posicionar o \$match corretamente e justificar a posição em termos de desempenho.
- Explicar por que \$lookup devolve array e o que o \$unwind faz com ele.
- Reconhecer quando \$facet economiza uma passada sobre os dados.

CONCEITOS

O pipeline é uma esteira

Cada estágio recebe o que o anterior devolveu e entrega ao seguinte. Isso significa que a ordem não é estilo, é semântica: o mesmo conjunto de estágios em ordem diferente produz resultado diferente. O cifrão antes do nome do campo significa "o valor deste campo"; sem ele, seria a string literal. No \$group, o _id define o critério de agrupamento, e _id: null junta tudo num resultado único.

ANALOGIA

Uma linha de produção. Cada posto recebe a peça do posto anterior, faz uma coisa só, e passa adiante. Trocar dois postos de lugar muda o produto final.

\$match antes é WHERE, \$match depois é HAVING

Colocado antes do \$group, o \$match filtra documentos crus — equivale ao WHERE do SQL, reduz o volume que entra na agregação e, o mais importante, ainda consegue usar índice. Colocado depois do \$group, filtra o resultado já agregado — equivale ao HAVING, e é a única posição em que se pode filtrar por um campo calculado, como uma média que não existia antes do agrupamento. A regra prática: empurre o \$match o mais para cima que a semântica permitir.

ERRO COMUM

Filtrar depois do \$group por um critério que poderia ter sido aplicado antes. O resultado é o mesmo, mas o custo não: agrupou-se documento que seria descartado, e sem índice.

\$avg ignora ausente, \$sum: 1 conta todo mundo

O \$avg desconsidera documentos em que o campo não existe, em vez de tratá-los como zero. É o comportamento desejável: uma média de notas não deve ser puxada para baixo por alunos que ainda não têm nota lançada. Já o \$sum: 1 conta o documento independentemente de ter ou não o campo, porque está somando a constante 1, não o campo. Confundir os dois produz relatório errado que parece certo.

ERRO COMUM

Calcular média dividindo \$sum do campo por \$sum: 1. O numerador ignora ausentes e o denominador não, então a média sai menor que a real.

Ordem dos branches no \$switch

O \$switch avalia os branches em sequência e o primeiro caso verdadeiro vence — os demais nem são testados. Numa classificação por faixas, isso obriga a ir da condição mais restritiva para a mais frouxa. Se { \$gte: 7 } vier antes de { \$gte: 9 }, a nota 9,7 será classificada como B, porque satisfaz o primeiro teste. Documentos que não satisfazem nenhum branch caem no default.

ERRO COMUM

Escrever as faixas em ordem crescente, por parecer mais natural de ler, e obter classificação sempre colada na faixa mais baixa.

\$lookup devolve array, sempre

O \$lookup faz o equivalente a um LEFT OUTER JOIN, mas o resultado vai para um campo no formato de array, mesmo quando apenas um documento casou — e um array vazio quando nenhum casou. Para achatar isso no formato de junção do SQL, usa-se o \$unwind, que produz um documento por item do array. Atenção: por padrão o \$unwind descarta documentos cujo array veio vazio, o que silenciosamente transforma o LEFT JOIN em INNER JOIN. Para preservá-los, é preciso preserveNullAndEmptyArrays: true.

ERRO COMUM

Fazer \$lookup seguido de \$unwind e perceber depois que os registros sem correspondência sumiram do relatório — sem nenhuma mensagem indicando isso.

\$facet: vários pipelines, uma passada

O \$facet executa múltiplos pipelines independentes sobre a mesma entrada e devolve todos os resultados num único documento. Serve para painéis, em que se quer contagem total, máximo e distribuição ao mesmo tempo, sem varrer a coleção três vezes.

ANALOGIA

É a mesma matéria-prima entrando em três linhas de produção paralelas, em vez de passar três vezes pela mesma linha.

NA PRÁTICA

A média que o \$avg calcula corretamente

```
db.estudantes.aggregate([
  { $match: { cidade: "São Paulo" } },
  { $group: { _id: "$cidade", media_nota: { $avg: "$nota" } } }
]);
```

SAÍDA REAL

```
[ { _id: "São Paulo", media_nota: 6.5 } ]
```

6,5 é a nota de Ricardo Mendes, o único estudante de São Paulo. Nos agrupamentos por curso, as médias não foram estragadas por Carlos, Ana e Lucas, que não têm o campo nota — o \$avg simplesmente não os considerou.

GABARITO.md, Parte 12, EX 12.2

Classificação por faixas: a ordem decide o resultado

```
db.estudantes.aggregate([
  { $project: {
    nome: 1, nota: 1,
    conceito: { $switch: {
      branches: [
        { case: { $gte: ["$nota", 9] }, then: "A" },
        { case: { $gte: ["$nota", 7] }, then: "B" }
      ],
      default: "C"
    } }
  } }
]);
```

Invertendo as duas linhas do branches, a nota 9,7 passa a receber B, porque o teste \$gte: 7 é verdadeiro e vence primeiro. Quem não tem nota cai no default e recebe C — o que pode ou não ser o desejado, e precisa ser decisão consciente.

GABARITO.md, Parte 12, EX 12.3

WHERE e HAVING no mesmo pipeline

```
// posição 1 — equivale ao WHERE: filtra documento cru, usa índice
{ $match: { cidade: "Brasília" } },
{ $group: { _id: "$curso", media_nota: { $avg: "$nota" } } }

// posição 2 — equivale ao HAVING: só aqui media_nota existe
{ $group: { _id: "$curso", media_nota: { $avg: "$nota" } } },
{ $match: { media_nota: { $gt: 8 } } }
```

O segundo \$match não poderia estar antes: media_nota é um campo criado pelo \$group e não existe nos documentos originais. Já o primeiro poderia estar depois — funcionaria, mas agruparia documentos que seriam descartados em seguida, e sem poder usar índice.

GABARITO.md, Parte 12, EX 12.5

FECHAMENTO

O pipeline é a ferramenta analítica do MongoDB, e quase toda decisão de desempenho nele se resume a uma pergunta: quanto dado consigo descartar antes do estágio caro? Essa pergunta leva diretamente ao próximo módulo, porque descartar cedo com eficiência depende de índice.

PARA DISCUTIR EM SALA

1. Por que o `$match` depois do `$group` não consegue usar índice?
2. Você precisa da média de notas e da contagem de alunos por curso. Vale um `$facet` ou dois pipelines separados?
3. Depois de um `$lookup` seguido de `$unwind`, seu relatório perdeu 12 registros. O que provavelmente aconteceu?

MongoDB: indexação e desempenho

Índice é a única estrutura em que se paga escrita para ganhar leitura. Saber criar é fácil; a competência real está em saber quando não criar — e em medir o efeito em vez de supor.

OBJETIVOS DE APRENDIZAGEM

- Ler a saída do explain e distinguir COLLSCAN de IXSCAN com FETCH.
- Usar totalDocsExamined como métrica de qualidade do índice, em vez do tempo de execução.
- Aplicar a regra do prefixo à esquerda para decidir a ordem dos campos num índice composto.
- Enumerar os casos em que criar índice piora o sistema.

CONCEITOS

COLLSCAN, IXSCAN e FETCH

COLLSCAN significa collection scan: o servidor leu todos os documentos, um a um, comparando o campo. Com índice, aparecem dois estágios aninhados: o IXSCAN percorre a estrutura do índice e encontra as chaves que casam; o FETCH usa os endereços guardados nessas chaves para buscar os documentos completos no disco. São dois estágios porque o índice guarda apenas o campo indexado e o ponteiro — não o documento inteiro.

ANALOGIA

COLLSCAN é folhear o livro página por página procurando uma palavra. IXSCAN é consultar o índice remissivo no fim; FETCH é ir até as páginas que o índice apontou.

Meça totalDocsExamined, não o tempo

Em coleção pequena, o tempo de execução com e sem índice é indistinguível — e às vezes o índice sai mais lento, porque consultá-lo e depois buscar os documentos custa mais do que varrer oito registros. O número que revela a qualidade do índice é totalDocsExamined: quantos documentos o servidor precisou abrir para responder. A meta é que ele fique próximo de nReturned. Uma razão de 1:1 é um índice perfeito para aquela consulta; 1000:1 é um índice que não está servindo.

ERRO COMUM

Benchmarkar índice numa base de teste com poucos documentos, concluir que "não fez diferença" e não criar o índice em produção, onde a coleção tem milhões de registros.

A regra do prefixo à esquerda

Um índice composto { cidade: 1, nota: -1 } atende consultas por cidade, e por cidade combinada com nota — inclusive entregando o resultado já ordenado por nota, sem ordenação em memória. Mas não atende consulta apenas por nota. O índice está ordenado primeiro por cidade; buscar só por nota exigiria varrer todos os blocos de cidade. Por isso a ordem dos campos num índice composto é decisão de projeto, não detalhe de escrita.

ANALOGIA

Uma lista telefônica ordenada por sobrenome e depois por nome. Dá para achar todos os Silva, e dá para achar Silva, João. Não dá para achar todos os Joãos sem ler a lista inteira.

Índice único: a integridade é do servidor

Um índice com { unique: true } faz o servidor recusar a escrita de valor repetido, devolvendo o erro E11000 duplicate key error, e o documento não é gravado. A validação acontece no servidor, no momento da escrita — não é convenção da aplicação, é garantia. Dois detalhes operacionais: não se converte um índice existente em único, é preciso removê-lo e recriar; e se a coleção já contiver valores repetidos, é a criação do índice que falha.

ERRO COMUM

Confiar em validação apenas na camada de aplicação. Duas instâncias do serviço gravando ao mesmo tempo passam pela validação da aplicação e produzem duplicata; só o índice único no servidor impede isso.

Quando o índice piora o sistema

Coleção pequena: varrer sai mais barato que consultar o índice e buscar os documentos. Campo de baixa cardinalidade, como um booleano ativo: o índice aponta para metade da base, não elimina quase nada, e o planejador tende a ignorá-lo. Carga de escrita intensa: cada índice precisa ser atualizado em todo insert, update e delete — cinco índices são cinco estruturas a manter por gravação. Campo que nunca aparece em filtro, ordenação ou \$lookup: ocupa disco e memória sem devolver nada. E o índice redundante: tendo { curso: 1, idade: -1 }, criar { curso: 1 } é desperdício, porque o composto já atende busca por curso sozinho.

ERRO COMUM

Criar um índice para cada campo "por precaução". O resultado é escrita lenta, uso de memória alto e um planejador com opções demais para avaliar.

NA PRÁTICA

O efeito do índice medido, não suposto

```
// antes de criar o índice
db.estudantes.find({ cidade: "Brasília" }).explain("executionStats");

db.estudantes.createIndex({ cidade: 1 });

// depois
db.estudantes.find({ cidade: "Brasília" }).explain("executionStats");
```

SAÍDA REAL

```
antes: stage: COLLSCAN  totalDocsExamined: 8  totalKeysExamined: 0
depois: stage: FETCH -> IXSCAN  totalKeysExamined: deixou de ser zero
```

O tempo praticamente não mudou, e com oito documentos isso é esperado. O que mudou foi `totalDocsExamined`, que caiu de oito para apenas os documentos que interessam. É esse número que se projeta para uma coleção de milhões.

GABARITO.md, Parte 13, EX 13.2

Um índice que serve filtro e ordenação de uma vez

```
db.estudantes.createIndex({ cidade: 1, nota: -1 });

db.estudantes.find({ cidade: "Brasília" }).sort({ nota: -1 });
```

Esse índice atende a consulta inteira: filtra por cidade e já devolve na ordem de nota decrescente, dispensando ordenação em memória. Note que a direção declarada no índice (-1) casa com a direção pedida no `sort`. O mesmo índice não serviria para uma busca apenas por nota.

GABARITO.md, Parte 13, EX 13.3

O servidor recusa a duplicata

```
db.cursos.createIndex({ nome: 1 }, { unique: true });

db.cursos.insertOne({ nome: "Data Science", duracao: "8 meses", modalidade: "EAD" });
```

SAÍDA REAL

```
MongoServerError: E11000 duplicate key error collection: dbNoSQLBD.cursos index: nome_1
```

O documento não foi gravado. Quem barrou foi o servidor, na escrita — uma garantia de integridade que nenhuma validação de aplicação consegue oferecer sob concorrência.

GABARITO.md, Parte 13, EX 13.4

FECHAMENTO

O índice B-tree do MongoDB resolve com elegância igualdade, faixa e ordenação. O que ele não resolve é relevância em texto livre: ele sabe dizer se um campo é igual a "banco de dados", mas não sabe dizer quais documentos são os mais relevantes para essa expressão, nem encontrar "bancos de dados" no meio de um parágrafo. É exatamente esse limite que justifica a existência do Elasticsearch, e é onde a aula vira.

PARA DISCUTIR EM SALA

1. Sua consulta retorna 50 documentos e o explain mostra totalDocsExamined: 40000. O que isso diz sobre o índice?
2. Você tem { curso: 1, idade: -1 }. Que consultas esse índice atende e que consultas ele não atende?
3. Um sistema recebe 10 mil escritas por segundo e faz uma consulta analítica por dia. Vale indexar o campo dessa consulta?
4. Por que não se pode converter um índice existente em índice único?

Elasticsearch: arquitetura e o índice invertido

O MongoDB responde rápido a "qual documento tem cidade igual a Brasília". Nenhum índice B-tree responde bem a "quais documentos falam sobre bancos de dados, ordenados por relevância". Essa segunda pergunta exige inverter a estrutura de dados.

OBJETIVOS DE APRENDIZAGEM

- Explicar cluster, node, index, shard e replica, e diagnosticar um cluster em estado yellow.
- Descrever o índice invertido e contrastá-lo com o B-tree do MongoDB.
- Prever o resultado de um analyzer sobre um texto e explicar por que match e term diferem.
- Justificar a escolha entre must e filter em termos de score e cache.
- Ler um explain de BM25 e identificar as três forças que compõem o score.

CONCEITOS

Cluster, node, shard e replica

Um cluster é um conjunto de nós que se coordenam. Um índice é dividido em shards, que são índices Lucene completos e independentes — é o shard que permite distribuir um índice grande demais para uma máquina. Cada shard primário pode ter réplicas, que servem a dois propósitos ao mesmo tempo: tolerância a falha e capacidade de leitura, já que consultas podem ser atendidas pela réplica. A regra que explica o estado do cluster: o Elasticsearch nunca aloca uma réplica no mesmo nó do seu primário, porque isso não protegeria contra nada.

ERRO COMUM

Interpretar o estado yellow como defeito. Yellow significa que todos os shards primários estão ativos e alguma réplica não foi alocada — o cluster está plenamente funcional, apenas sem redundância. Red é que significa primário faltando, ou seja, dado inacessível.

O índice invertido

Um índice comum vai do documento para o conteúdo. O índice invertido faz o contrário: para cada termo do vocabulário, guarda a lista de documentos em que ele aparece — a posting list — junto com frequência e posição. Responder "quais documentos contêm banco" vira uma consulta direta ao termo, sem varrer documento algum. E como as listas estão ordenadas, uma busca por dois termos é a interseção de duas listas ordenadas, operação linear no tamanho delas. É essa estrutura, implementada pelo Lucene, que está por baixo de todo o Elasticsearch.

ANALOGIA

O índice remissivo no fim de um livro técnico: não guarda o texto, guarda em que páginas cada termo aparece. Procurar "normalização" no remissivo é instantâneo; procurar folheando não é.

O analyzer decide o que vira termo

Antes de entrar no índice invertido, o texto passa pelo analyzer, que faz três coisas em sequência: filtros de caractere, tokenização (quebra em termos) e filtros de token (minúsculas, remoção de stopwords, stemming). O ponto crucial é que o mesmo analyzer é aplicado na indexação e na busca — é essa simetria que faz a consulta encontrar o documento. Quando os dois divergem, a busca falha silenciosamente.

ERRO COMUM

Supor que o analyzer standard remove acentos. Ele não remove: faz minúsculas e separa por limites de palavra, mas "joão" e "joao" permanecem termos distintos. Remover acento exige o filtro asciifolding num analyzer customizado.

match analisa, term não

O match passa o texto da consulta pelo mesmo analyzer do campo e busca os termos resultantes — é a busca full-text. O term procura o valor exatamente como foi escrito, sem análise alguma. Daí a regra prática: match sobre campo text, term sobre campo keyword. Usar term sobre um campo text quase sempre devolve vazio, porque o valor original ("São Paulo") nunca foi armazenado como termo — o que está no índice são os tokens ("são", "paulo").

ERRO COMUM

Escrever `{ "term": { "cidade": "São Paulo" } }` sobre um campo text e receber zero resultados, concluindo que o dado não existe. O correto é `cidade.keyword`, ou trocar para match.

must pontua, filter apenas seleciona

Dentro de uma bool query, as cláusulas em must contribuem para o score de relevância; as cláusulas em filter apenas decidem se o documento entra ou não, sem influenciar a ordenação. A diferença é de custo: como filter não calcula score, o Elasticsearch pode reaproveitar o resultado em cache de bitset entre consultas. Critérios binários — faixa de data, categoria, status — pertencem a filter. Só o que expressa "quão bem este documento responde à pergunta" pertence a must.

ERRO COMUM

Colocar tudo em must por hábito. O sistema funciona, mas paga cálculo de score para condições que são simples sim-ou-não e perde o cache.

BM25: as três forças do score

O BM25 combina três fatores. A frequência do termo no documento (TF) aumenta o score, mas com saturação: a décima ocorrência acrescenta muito menos que a segunda. A frequência inversa nos documentos (IDF) premia termos raros — um termo presente em todos os documentos não discrimina nada e recebe peso baixo. E a normalização por tamanho penaliza documentos longos, porque conter o termo num texto de dez mil palavras é menos significativo do que contê-lo em vinte. A fórmula do IDF aparece literalmente no explain: $\log(1 + (N - n + 0.5) / (n + 0.5))$.

ANALOGIA

Numa biblioteca, encontrar "de" num livro não diz nada — está em todos. Encontrar "hidrodinâmica" diz muito. E encontrá-la num panfleto de duas páginas diz mais do que encontrá-la numa enciclopédia de mil.

Near real-time: o documento não aparece na hora

O Lucene grava em segmentos imutáveis. Um documento indexado fica num buffer em memória e só se torna pesquisável quando ocorre o refresh, que por padrão roda a cada segundo e cria um novo segmento. Por isso o Elasticsearch é near real-time e não real-time. Segmentos são imutáveis, o que traz uma consequência importante: atualizar um documento não altera nada — grava-se uma versão nova e marca-se a antiga como deletada. O espaço e as estatísticas da versão antiga só somem quando ocorre um merge de segmentos.

ERRO COMUM

Indexar um documento em teste automatizado e consultá-lo na linha seguinte, obtendo zero resultados. A correção é usar `?refresh=true` na escrita — em teste. Em produção, forçar refresh a cada escrita destrói o desempenho, porque cria um segmento por operação.

NA PRÁTICA

Diagnosticar um cluster yellow

```
GET _cat/shards/meu_indice-000001,transacoes-000001?v
```

SAÍDA REAL

index	shard	prirep	state	docs	store	node
transacoes-000001	0	p	STARTED	2	18kb	96ef8f7b41d1
transacoes-000001	0	r	UNASSIGNED			
meu_indice-000001	0	p	STARTED	2	17.7kb	96ef8f7b41d1
meu_indice-000001	0	r	UNASSIGNED			

Os primários (p) estão STARTED; as réplicas (r) estão UNASSIGNED e sem nó atribuído. Com um único nó não há onde alocá-las, porque o Elasticsearch se recusa a pôr a réplica junto do primário. O cluster funciona por completo — apenas não sobrevive à perda do nó.

PARTE2-EVIDENCIAS.md, seção 12.1

O que o analyzer realmente faz — e o que não faz

```
POST _analyze
{ "analyzer": "standard", "text": "JOÃO, da Silva; em BRASÍLIA!" }
```

SAÍDA REAL

```
tokens: [ "joão", "da", "silva", "em", "brasília" ]
```

A pontuação sumiu e tudo virou minúscula — mas os acentos permaneceram. O roteiro do laboratório afirma que isso explica "por que match joao encontra João". Foi testado no cluster: match "joao" retorna zero resultados, enquanto "joão" e "JOÃO" retornam um. O analyzer standard explica o lowercase, não a insensibilidade a acento.

PARTE2-EVIDENCIAS.md, seção 12.3; teste de acentuação executado em 08/09/2026

O IDF do BM25, com os números do laboratório

```
GET meu_indice/_search
{ "query": { "match": { "nome": "João" } }, "explain": true }
```

SAÍDA REAL

```
weight(nome:joão in 0) [PerFieldSimilarity] = 0.4700036
boost = 2.2
idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) = 0.47000363
  n, number of documents containing term = 2
  N, total number of documents with field = 3
tf = 0.45454544
```

Repare no detalhe: o índice tinha apenas 2 documentos vivos, mas o BM25 reporta $N = 3$. A causa é a imutabilidade dos segmentos — a Parte 10 fez um update, que gravou versão nova e marcou a antiga como deletada, e as estatísticas contam o documento deletado até o merge. Isso foi confirmado em experimento controlado: um índice com 2 documentos, após um único update, passa a reportar $N = 3$ com `docs.deleted = 1`.

PARTE2-EVIDENCIAS.md, seção 12.6; experimento de verificação em 09/09/2026

FECHAMENTO

O índice invertido é o que torna a busca por relevância viável, e o BM25 é o que a torna útil. Mas nada disso funciona se os campos não forem declarados com o tipo certo — e é aí que a modelagem no Elasticsearch se revela mais rígida do que a do MongoDB, não menos.

PARA DISCUTIR EM SALA

1. Se réplica serve para tolerância a falha, por que ela também aumenta a capacidade de leitura?
2. Um campo tem o termo "banco" em todos os documentos do índice. Que score de IDF ele recebe, e o que isso significa na prática?
3. Por que forçar refresh a cada escrita destrói o desempenho em produção?
4. Como você faria uma busca que encontrasse "João" digitando "joao"?

Elasticsearch: mapping, ingestão e consulta

No MongoDB, gravar primeiro e pensar no esquema depois costuma dar certo. No Elasticsearch, essa ordem custa caro: o mapping precisa existir antes do índice nascer, porque mudar o tipo de um campo exige reindexar tudo.

OBJETIVOS DE APRENDIZAGEM

- Escolher entre text e keyword com base na operação pretendida.
- Justificar a existência de index template no ciclo de vida de índices que rotacionam.
- Explicar o papel do alias e do `is_write_index` numa estratégia de rollover.
- Comparar a Bulk API com inserções individuais em termos de custo.
- Traduzir uma agregação do Elasticsearch para o `$group` do MongoDB e vice-versa.

CONCEITOS

text e keyword resolvem problemas opostos

Um campo text passa pelo analyzer, é quebrado em termos e serve para busca full-text — mas por isso mesmo não pode ser usado para ordenação, agregação exata ou filtro por valor integral. Um campo keyword é armazenado como um único termo, sem análise, e serve exatamente para o que o text não serve. Como quase todo campo textual precisa das duas coisas, o padrão é declarar o multi-field: o campo é text e ganha um sub-campo .keyword. É por isso que aparece cidade para buscar e cidade.keyword para agregar.

ERRO COMUM

Tentar ordenar ou agregar por um campo text e receber o erro "Fielddata is disabled on text fields by default". A mensagem sugere habilitar fielddata, o que é quase sempre a decisão errada: consome memória proporcional ao vocabulário. O certo é usar o sub-campo keyword.

ignore_above: o corte silencioso

O sub-campo keyword costuma vir com ignore_above: 256, o que significa que valores com mais de 256 caracteres simplesmente não são indexados naquele sub-campo. O documento é aceito e o campo text continua pesquisável, mas o valor some das agregações e dos filtros exatos. É um comportamento defensivo — evita termos gigantes no índice — que se manifesta como dado faltando em relatório.

ERRO COMUM

Agregar por um campo .keyword e notar que alguns registros não aparecem em bucket algum, sem nenhum erro que indique o motivo.

Index template: o mapping precede o índice

Índices que rotacionam por tempo — logs diários, por exemplo — nascem sozinhos quando o primeiro documento chega. Se não houvesse template, cada índice novo teria mapping inferido automaticamente, e bastaria um documento atípico para que um campo virasse texto num dia e número no outro. O template casa por padrão de nome e aplica settings, mappings e aliases a todo índice criado que corresponda ao padrão. Ele age na criação: alterar o template não muda índices que já existem.

ERRO COMUM

Corrigir o mapping no template e supor que os índices já criados foram corrigidos. Só valem para os próximos; os antigos precisam de reindex.

Alias e `is_write_index`

O alias é um nome estável que aponta para um ou mais índices físicos. A aplicação escreve e lê por meio dele e nunca precisa saber que por baixo existe `transacoes-000001`, depois `000002`, e assim por diante. Quando o alias aponta para vários índices, é preciso marcar um deles com `is_write_index: true` para dizer onde as escritas devem cair — leituras vão a todos, escritas a apenas um. É esse mecanismo que torna o rollover transparente.

ANALOGIA

É o encaminhamento de correspondência. O remetente continua escrevendo para o mesmo endereço; quem se muda é o destinatário, e a correspondência antiga continua acessível.

Bulk API: o custo está na viagem, não na escrita

Vinte inserções individuais são vinte requisições HTTP, com vinte handshakes, vinte roteamentos de coordenação e vinte respostas. Um único `_bulk` com vinte operações é uma requisição só: o nó coordenador agrupa as operações por shard e as despacha em lote. O ganho não vem de escrever mais rápido, vem de eliminar o custo por viagem. O mesmo raciocínio vale para o `insertMany` do MongoDB.

ERRO COMUM

Assumir que status 200 no `_bulk` significa que tudo deu certo. O bulk devolve 200 mesmo com falhas parciais — é obrigatório inspecionar o campo `errors` e o status de cada item.

Agregações: o mesmo conceito, outra sintaxe

A agregação `terms` cria um bucket por valor distinto do campo e conta os documentos de cada um — é o `$group` com `$sum: 1` do MongoDB. A `avg` calcula média — é o `$group` com `$avg`. O `size`: o no corpo da consulta pede que os documentos não sejam devolvidos, apenas os buckets, o que evita trafegar dados que ninguém vai ler. E, como no MongoDB, agregações operam sobre valores exatos: por isso o campo agregado precisa ser keyword.

ANALOGIA

São dois idiomas descrevendo a mesma operação. Quem entende agrupamento em um, entende no outro assim que traduz o vocabulário.

O template que garante o tipo antes do índice existir

```
PUT _template/transacoes_template
{
  "index_patterns": ["transacoes-*"],
  "settings": {
    "index.lifecycle.name": "politica_transacoes",
    "index.lifecycle.rollover_alias": "transacoes"
  },
  "mappings": { "properties": {
    "usuario": { "type": "text",
      "fields": { "keyword": { "type": "keyword", "ignore_above": 256 } } },
    "valor": { "type": "double" },
    "tipo": { "type": "keyword" },
    "data": { "type": "date",
      "format": "strict_date_optional_time||epoch_millis" }
  } }
}
```

SAÍDA REAL

```
{ "acknowledged": true }
```

Note as três decisões de modelagem: usuario é text com sub-campo keyword, porque precisa ser buscado e agregado; tipo é keyword puro, porque é categoria fechada e nunca será buscado por palavra solta; valor é double, o que habilita range e avg. Trocar tipo por text quebraria a agregação do próximo slide.

PARTE1-EVIDENCIAS.md, Parte 6

Alias com índice de escrita definido

```
PUT transacoes-000001
{ "aliases": { "transacoes": { "is_write_index": true } } }
```

SAÍDA REAL

```
{ "acknowledged": true, "shards_acknowledged": true, "index": "transacoes-000001" }
```

A partir daqui a aplicação usa apenas o nome transacoes. Quando o rollover criar o 000002 e transferir a marca de escrita, nenhuma linha de código da aplicação muda — e as consultas continuam alcançando os dois índices.

PARTE1-EVIDENCIAS.md, Parte 7

Agregação por categoria, com os números reais do lab

```
GET transacoes/_search
{ "size": 0,
  "aggs": { "total_por_tipo": { "terms": { "field": "tipo" } } } }
```

SAÍDA REAL

```
"buckets": [
  { "key": "compra", "doc_count": 12 },
  { "key": "venda", "doc_count": 10 }
]
```

Vinte e dois documentos, dois buckets. Isso só funciona porque tipo foi declarado keyword no template. Se fosse text, os valores teriam sido tokenizados e a agregação falharia. O equivalente em MongoDB seria { \$group: { _id: "\$tipo", total: { \$sum: 1 } } }.

PARTE2-EVIDENCIAS.md, seção 12.9

Média sobre campo numérico

```
GET transacoes/_search
{ "size": 0, "aggs": { "valor_medio": { "avg": { "field": "valor" } } } }
```

SAÍDA REAL

```
"hits": { "total": { "value": 22 } },
"aggregations": { "valor_medio": { "value": 302.26590909090913 } }
```

A média de 22 transações. O size: o fez com que nenhum documento fosse devolvido — apenas o resultado agregado. Sem ele, o Elasticsearch mandaria os 10 primeiros documentos junto, de graça, para ninguém ler.

PARTE2-EVIDENCIAS.md, seção 12.9

FECHAMENTO

Modelagem no Elasticsearch é uma decisão antecipada: text ou keyword, com ou sem sub-campo, template antes do índice. Feita a modelagem, a busca lexical está resolvida — mas ela só encontra o que foi escrito com as mesmas palavras. Buscar por significado exige outra estrutura.

PARA DISCUTIR EM SALA

1. Você precisa agregar por país e também buscar por nome de país digitado parcialmente. Como declara o campo?
2. Um `_bulk` de mil documentos retornou 200. Por que isso não basta para concluir que tudo foi indexado?
3. Se alterar o template não corrige índices existentes, como você corrigiria o mapping de um índice em produção?
4. Por que agregação exige keyword, se o campo text contém a mesma informação?

Busca vetorial, busca híbrida e RAG

O BM25 encontra documentos que usam as mesmas palavras da consulta. Quem pergunta "qual a capital do Brasil" e tem no acervo um texto que diz "Brasília é a sede do governo federal" não recebe nada — não há palavra em comum. Resolver isso exige representar significado, não grafia.

OBJETIVOS DE APRENDIZAGEM

- Explicar o que é um embedding e por que proximidade vetorial aproxima significado.
- Declarar um campo `dense_vector` e justificar a escolha do número de dimensões.
- Distinguir `script_score` de `knn` em termos de exatidão, custo e faixa de score.
- Descrever o pipeline de RAG e justificar por que se indexam chunks, não documentos.

CONCEITOS

Embedding: significado como coordenada

Um embedding é um vetor de números reais produzido por um modelo treinado, de modo que textos com sentido próximo recebam vetores próximos no espaço. O modelo não guarda as palavras: guarda uma posição num espaço de centenas de dimensões, onde a distância codifica relação semântica. É isso que permite recuperar "Brasília é a sede do governo" a partir de "capital do Brasil", sem nenhuma palavra em comum.

ANALOGIA

Um mapa de cidades. Nada no nome "Campinas" indica que ela está perto de São Paulo — quem informa isso são as coordenadas. O embedding faz o mesmo com significado: transforma sentido em posição, e semelhança em proximidade.

`dense_vector` e a rigidez das dimensões

O campo `dense_vector` armazena o vetor e exige declarar `dims`, o número de dimensões. Esse número não é escolha livre: precisa coincidir exatamente com a saída do modelo usado. O `all-MiniLM-L6-v2` produz 384 dimensões; outros modelos produzem 768, 1024 ou 1536. Documento com dimensão diferente da declarada é rejeitado, e trocar de modelo depois obriga a reindexar tudo, porque vetores de modelos diferentes não são comparáveis entre si — nem quando têm o mesmo número de dimensões.

ERRO COMUM

Comparar vetores gerados por modelos distintos. Os números têm o mesmo formato e a operação de cosseno funciona sem erro, mas o resultado não significa nada: são espaços diferentes.

Similaridade de cosseno

O cosseno entre dois vetores mede o ângulo entre eles, ignorando o comprimento: $\text{sim}(A,B) = (A \cdot B) / (||A|| \times ||B||)$. O valor vai de 1, para vetores na mesma direção, passando por 0, para vetores ortogonais, até -1, para direções opostas. Ignorar o comprimento é desejável em texto, porque um documento longo e um curto sobre o mesmo assunto devem ser considerados próximos. Como o Elasticsearch exige score não negativo, o `script_score` soma 1,0 ao cosseno, deslocando a faixa para 0 a 2.

ANALOGIA

Duas pessoas apontando para o mesmo ponto do horizonte. O cosseno mede se os braços apontam na mesma direção — não importa se um braço é mais comprido que o outro.

`script_score` é exato; knn é aproximado

O `script_score` avalia todos os documentos que passam pela query interna, calculando o cosseno sobre os vetores originais em ponto flutuante. É exato e custa $O(n)$ — aceitável em milhares de documentos, inviável em milhões. A query `knn` do Elasticsearch 8 percorre um grafo HNSW, que encontra vizinhos prováveis em tempo sublinear, sem examinar todos os candidatos. Duas consequências: o resultado pode não ser o exato (por isso "aproximado"), e a faixa de score é outra — para cosseno, o `knn` usa $(1 + \cos) / 2$, que vai de 0 a 1.

ERRO COMUM

Comparar diretamente o score de um `script_score` com o de um `knn` e concluir que um "achou melhor" que o outro. As escalas são diferentes por definição; comparar exige converter.

Quantização: o `int8` que o 8.x aplica sozinho

No Elasticsearch 8.x, um `dense_vector` indexado ganha por padrão `index_options` do tipo `int8_hnsw`: os vetores são quantizados de 32 para 8 bits por dimensão, reduzindo memória em cerca de quatro vezes. O preço é um erro pequeno de precisão, que aparece como diferença na terceira ou quarta casa decimal do score. Em recuperação isso raramente altera o ranking, mas explica por que a conta do `knn` não fecha exatamente com a fórmula teórica.

ANALOGIA

É arredondar todo preço para a dezena mais próxima antes de somar. O total continua servindo para decidir, mas não bate no centavo com a soma exata.

Busca híbrida

Lexical e vetorial erram de formas diferentes. O BM25 falha com sinônimos e paráfrases; a busca vetorial falha com termos exatos, códigos de produto, nomes próprios e números — coisas que o embedding tende a diluir. Combinar as duas cobre os dois pontos cegos. No laboratório a combinação foi feita com `script_score`, em que o `match` filtra e o cosseno reordena. Em produção, a técnica atual é RRF (Reciprocal Rank Fusion), que funde os dois rankings sem precisar normalizar escalas incompatíveis.

ERRO COMUM

Somar diretamente score de BM25 com score de cosseno. O BM25 não tem limite superior e varia com o corpus; o cosseno é limitado. A soma é dominada arbitrariamente por um dos dois.

RAG e a razão do chunking

Num pipeline de RAG, documentos são divididos em trechos, cada trecho vira um embedding, tudo é indexado, e a pergunta do usuário — também convertida em vetor — recupera os k trechos mais próximos, que entram no prompt do modelo de linguagem. Indexa-se trecho e não documento inteiro por duas razões: um vetor único para um documento longo é a média de assuntos demais e não se aproxima de nenhum, e o contexto entregue ao modelo precisa caber na janela. O tamanho do chunk é um compromisso — pequeno demais perde contexto, grande demais dilui o significado.

ERRO COMUM

Tratar o RAG como solução completa e ignorar a qualidade da recuperação. Se os trechos recuperados forem ruins, o modelo responde com confiança sobre a base errada.

NA PRÁTICA

O que o Elasticsearch 8 acrescenta ao mapping sem pedir

```
PUT documentos_vetoriais
{ "mappings": { "properties": {
  "texto":      { "type": "text" },
  "embedding": { "type": "dense_vector", "dims": 384 }
} } }

GET documentos_vetoriais/_mapping
```

SAÍDA REAL

```
"embedding": {
  "type": "dense_vector", "dims": 384,
  "index": true,
  "similarity": "cosine",
  "index_options": { "type": "int8_hnsw", "m": 16, "ef_construction": 100 }
}
```

Foram declarados dois atributos e voltaram cinco. O roteiro do laboratório, escrito para o Elasticsearch 7.x, afirma que index e similarity "não devem ser usados" — no 8.15 eles são aplicados automaticamente. Isso muda a prática: o campo já nasce indexado em HNSW e aceita a query knn, que no 7.x não existia.

PARTE2-EVIDENCIAS.md, seção 13.2

Os dois caminhos, medidos lado a lado

```
// caminho A – exato, força bruta
POST indice_conhecimento/_search
{ "query": { "script_score": {
  "query": { "match_all": {} },
  "script": { "source": "cosineSimilarity(params.query_vector, 'embedding') + 1.0",
    "params": { "query_vector": [0.11, -0.40, 0.80, ...] } } } },
  "size": 3 }

// caminho B – aproximado, grafo HNSW
POST indice_conhecimento/_search
{ "knn": { "field": "embedding", "query_vector": [0.11, -0.40, 0.80, ...],
  "k": 3, "num_candidates": 100 } }
```

SAÍDA REAL

titulo	script_score	knn	cosseno exato	(1+cos)/2
Curitiba	1.99895	0.99936	0.998947	0.999473
Cerrado brasileiro	1.99894	0.99935	0.998938	0.999469
Capital do Brasil	1.99875	0.99924	0.998748	0.999374

Mesmo ranking pelos dois caminhos. O script_score bate exatamente com o cosseno calculado à mão ($1,99895 = 0,998947 + 1$). O knn fica levemente abaixo de $(1+\cos)/2$ — a diferença é o erro da quantização int8. Ou seja: as duas colunas não divergem por bug, divergem porque uma é exata e a outra é aproximada e comprimida.

PARTE2-EVIDENCIAS.md, seções 14.4 e EXTRA; verificação numérica em 09/09/2026

FECHAMENTO

Uma ressalva honesta sobre o laboratório: os vetores de 10 dimensões foram escritos à mão, não gerados por modelo. A mecânica do cosseno, do HNSW e da faixa de score é real e foi medida; a semântica, não — documentos "próximos" ali estavam próximos porque os números foram escolhidos assim. Com um modelo real de 384 dimensões, a mesma mecânica passa a carregar significado de verdade.

PARA DISCUTIR EM SALA

1. Por que o cosseno ignora o comprimento do vetor, e por que isso é desejável em texto?
2. Sua busca vetorial não encontra o produto "XPS-13-9370" que o usuário digitou. Por quê, e o que você faria?
3. Em que situação o knn devolveria um ranking diferente do script_score sobre os mesmos dados?
4. Se o chunk pequeno perde contexto e o grande dilui significado, como você decidiria o tamanho?

Os dois juntos: sistema de registro e camada de busca

Os dois laboratórios pareciam exercícios separados. Não eram: juntos, eles montaram uma arquitetura que aparece em quase todo sistema de porte — e que responde à pergunta de quando usar cada banco com um "os dois, em papéis diferentes".

OBJETIVOS DE APRENDIZAGEM

- Descrever o fluxo MongoDB → GELF → Logstash → Elasticsearch → Kibana e o papel de cada peça.
- Explicar por que o acoplamento entre os dois sistemas é unidirecional e o que isso garante.
- Avaliar a escolha de UDP para transporte de log, com suas vantagens e o que ela sacrifica.
- Aplicar o critério de decisão entre os dois sistemas a requisitos concretos.

CONCEITOS

O fluxo, peça por peça

O MongoDB escreve log na saída padrão, como qualquer processo. O driver de log GELF do Docker intercepta essa saída e a envia por UDP para a porta 12201. O Logstash escuta ali, aplica o pipeline configurado e escreve no Elasticsearch, num índice nomeado por data. O Kibana lê o Elasticsearch e apresenta. Nenhuma dessas peças conhece as outras além da vizinha imediata: trocar o Elasticsearch por outro destino exigiria mudar apenas o output do Logstash.

ANALOGIA

Uma esteira de triagem postal. Cada posto sabe de onde recebe e para onde entrega, e nenhum precisa conhecer o percurso inteiro.

Acoplamento de um lado só

O MongoDB não sabe que o Elasticsearch existe. Ele apenas emite log, e emitiria da mesma forma se ninguém estivesse escutando. Essa direção única é o que torna a arquitetura resiliente: derrubar toda a camada de busca não afeta a capacidade do banco de aceitar escritas. A recíproca não vale — sem a fonte, o índice derivado deixa de ser alimentado.

ERRO COMUM

Inverter a dependência, fazendo a aplicação escrever primeiro no Elasticsearch e depois no banco principal. Qualquer indisponibilidade da busca passa a derrubar a escrita.

UDP: a escolha que troca garantia por isolamento

GELF sobre UDP não estabelece conexão nem confirma entrega. A consequência boa é que o contêiner sobe e opera mesmo com o Logstash fora do ar — os pacotes são simplesmente descartados e nada bloqueia. A consequência ruim é que não há garantia: sob perda de rede ou sobrecarga, linhas de log somem sem aviso. Para telemetria é um compromisso razoável; para auditoria, não. Auditoria exige transporte confiável e confirmação de entrega.

ERRO COMUM

Usar o mesmo pipeline de log best-effort para trilha de auditoria com valor legal. O dia em que faltar a linha crítica, não haverá como saber se ela nunca existiu ou se foi perdida.

O índice de busca nunca é a fonte da verdade

Este é o princípio que organiza tudo. O Elasticsearch é um índice derivado: se for perdido, reconstrói-se reindexando a partir da fonte. Se a fonte for perdida, o índice vira uma coleção de afirmações sobre dados que não existem mais. Isso tem consequência prática de projeto: todo pipeline de indexação precisa ser reexecutável do zero, e a decisão de "gravar só no Elasticsearch porque é mais rápido" deve ser recusada por padrão.

ANALOGIA

O acervo e o catálogo da biblioteca. Perder o catálogo é trabalhoso; perder o acervo é irreversível.

O critério de escolha, sem falso dilema

A pergunta útil não é "qual banco é melhor", é "que pergunta este dado precisa responder, e com que garantia". Estado atual de um registro, com escrita confirmada e possibilidade de transação: MongoDB. Relevância em texto livre, agregação exploratória sobre volume, busca semântica: Elasticsearch. Quando um sistema precisa das duas coisas — e a maioria precisa — a resposta é os dois, com fluxo de sincronização explícito e a verdade morando em um só lugar.

ERRO COMUM

Escolher um único sistema para tudo e depois forçá-lo no papel para o qual não foi feito: implementar busca por relevância com regex no MongoDB, ou tratar o Elasticsearch como banco transacional.

NA PRÁTICA

O pipeline em treze linhas

```
# lab02-elastic/logstash/pipeline/logstash.conf
input {
  gelf { port => 12201 }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "mongodb-logs-%{+YYYY.MM.dd}"
  }
  stdout { codec => rubydebug }
}
```

SAÍDA REAL

health	status	index	docs.count
yellow	open	mongodb-logs-2026.09.08	160
yellow	open	mongodb-logs-2026.09.09	86

Dois índices, um por dia — a rotação vem do padrão de data no nome. É exatamente esse formato que justifica o index template do Módulo 6: índices que nascem sozinhos precisam de mapping declarado antes.

lab02-elastic/logstash/pipeline/logstash.conf; estado do cluster em 09/09/2026

A prova de que o acoplamento é de um lado só

```
# durante a reorganizacao do repositório, toda a camada de busca foi derrubada:
$ cd lab02-elastic && docker compose down
$ cd ../lab01-mongodb && docker compose up -d # MongoDB sobe normalmente

# e ao religar, o fluxo recomeça sozinho:
$ curl -s "localhost:9200/_cat/indices/mongodb-logs-*?v"
```

SAÍDA REAL

yellow	open	mongodb-logs-2026.09.09	86 docs
--------	------	-------------------------	---------

O MongoDB subiu com o Logstash fora do ar, sem erro e sem espera — porque GELF é UDP e não bloqueia. Os logs daquele intervalo foram perdidos, o que é aceitável para telemetria. Ao religar a camada de busca, a indexação recomeçou sem nenhuma intervenção.

Reorganização do repositório, 08/09/2026

A mesma pergunta nos dois sistemas

```
// MongoDB – quantos documentos por categoria
db.transacoes.aggregate([
  { $group: { _id: "$tipo", total: { $sum: 1 } } }
]);

// Elasticsearch – a mesma pergunta
GET transacoes/_search
{ "size": 0, "aggs": { "por_tipo": { "terms": { "field": "tipo" } } } }
```

SAÍDA REAL

```
MongoDB:      [ { _id: "compra", total: 12 }, { _id: "venda", total: 10 } ]
Elasticsearch: [ { key: "compra", doc_count: 12 }, { key: "venda", doc_count: 10 } ]
```

Mesma resposta, vocabulário diferente. A distinção entre os dois sistemas não está em agregar — os dois agregam bem. Está no que cada um faz com texto: só o Elasticsearch ordena por relevância, e só o MongoDB garante a escrita que originou o dado.

GABARITO.md Parte 12 e PARTE2-EVIDENCIAS.md seção 12.9

FECHAMENTO

A competência que esta aula pretende deixar não é sintaxe de nenhum dos dois — isso está na documentação. É saber formular a pergunta certa diante de um requisito: que garantia este dado exige, que pergunta ele precisa responder, e onde mora a verdade. Respondido isso, a escolha da ferramenta é quase automática.

PARA DISCUTIR EM SALA

1. Se o Elasticsearch é descartável e reconstruível, por que fazer backup dele?
2. Sua aplicação precisa que uma alteração no MongoDB apareça na busca em menos de um segundo. O pipeline de log serve? O que você usaria?
3. Que perguntas você faria a um time que propõe migrar do PostgreSQL para o MongoDB?
4. Onde mora a fonte da verdade no sistema em que você trabalha ou estuda, e como você sabe disso?

Anexo A — O que o roteiro não conta

Problemas que aconteceram de verdade ao executar os dois laboratórios nesta máquina. Cada um traz o sintoma, a causa raiz e a lição generalizável.

A imagem mongo:4.4 do roteiro não sobe em kernel moderno MONGODB

SINTOMA	O contêiner entra em laço de reinício. O log repete sempre a mesma linha e o serviço nunca fica disponível.
CAUSA RAIZ	Bug SERVER-121912: versões antigas do MongoDB têm incompatibilidade conhecida com kernels Linux 6.19 ou superiores. O OrbStack roda kernel 7.0.14, verificado com <code>docker run --rm alpine uname -r</code> . O roteiro pressupõe a VM Ubuntu, cujo kernel é mais antigo.
DIAGNÓSTICO	Ler o log do contêiner em vez de olhar apenas o status. O laço de reinício não diz a causa; a mensagem no log diz.
CORREÇÃO	Trocar para mongo:8.2, que já contém a correção. Efeito colateral bem-vindo: essa imagem traz o mongosh embutido, o que torna a Parte 2 do roteiro (instalação via apt) desnecessária.

```
MongoDB cannot start: Linux kernel versions 6.19 and newer has a known incompatibility with this version of MongoDB.
```

Lição: A versão fixada num roteiro carrega premissas sobre o ambiente que o roteiro não declara. Quando a versão indicada não sobe, o primeiro passo é ler o log, não trocar de ferramenta.

Elasticsearch 8.x exige HTTPS e token por padrão ELASTICSEARCH

SINTOMA	Seguindo o roteiro, o Kibana não conecta e as chamadas em <code>http://localhost:9200</code> são recusadas.
CAUSA RAIZ	A partir do 8.0, a segurança vem ligada por padrão: TLS obrigatório, senha gerada na primeira subida e token de inscrição para o Kibana. O roteiro foi escrito para o 7.x, onde tudo isso era opcional.
DIAGNÓSTICO	Ler o log do contêiner do Elasticsearch na primeira subida — é ali que a senha gerada e o token de inscrição aparecem, uma única vez.
CORREÇÃO	Para o laboratório, definir <code>xpack.security.enabled=false</code> no compose. Isso desliga autenticação, autorização por roles, TLS entre nós e no cliente, e a auditoria.

```
lab02-elastic/docker-compose.yml — environment: xpack.security.enabled=false
```

Lição: Desligar segurança para estudar é legítimo e deve ser uma decisão consciente e registrada. O risco real não é o ambiente de laboratório: é essa configuração viajar para produção dentro de um compose copiado.

O roteiro afirma que o analyzer standard remove acentos. Não remove. ELASTICSEARCH

SINTOMA	A seção 12.3 do roteiro diz que o analyzer explica "por que match joao encontra João". Quem confia na afirmação e implementa busca contando com isso descobre em produção que não funciona.
CAUSA RAIZ	O analyzer standard aplica lowercase e separa por limites de palavra, mas não faz transliteração. Os tokens gerados preservam o acento, e "joao" e "joão" permanecem termos distintos no índice invertido.
DIAGNÓSTICO	Rodar POST _analyze com o texto e inspecionar os tokens, em vez de supor o comportamento. Depois confirmar com uma busca real, comparando as duas grafias.
CORREÇÃO	Declarar um analyzer customizado com o filtro asciifolding na cadeia, e aplicá-lo tanto na indexação quanto na busca. O padrão comum é manter o campo original e acrescentar um sub-campo folded para busca insensível a acento.

```
match "João" -> 1 hit      |      match "joao" -> 0 hits
match "joão" -> 1 hit     |      match "JOÃO" -> 1 hit
tokens de "JOÃO, da Silva; em BRASÍLIA!": [joão, da, silva, em, brasilíia]
```

Lição: Material didático também erra. O custo de verificar uma afirmação sobre o motor é um comando; o custo de não verificar é uma busca que falha silenciosamente para uma parte dos usuários.

Um índice com política ILM inexistente é criado sem erro — e falha depois, em silêncio

ELASTICSEARCH

SINTOMA	O template referencia as políticas minha_politica e politica_transacoes, que nunca foram criadas. O template é aceito, o índice é criado, tudo parece correto. Meses depois, os índices não rotacionam nem expiram, e ninguém sabe por quê.
CAUSA RAIZ	A validação de index.lifecycle.name não acontece na criação. O setting é apenas gravado. Quem resolve o nome da política é o serviço de ILM, que roda periodicamente em segundo plano — e é lá, de forma assíncrona, que o erro aparece.
DIAGNÓSTICO	GET <índice>/_ilm/explain. Foi o que revelou o problema num teste controlado: o índice aparece como managed: true, mas com step: ERROR e a razão explícita.
CORREÇÃO	Criar a política antes do template, ou remover as duas linhas de lifecycle do template se não houver política. E monitorar índices em estado de erro de ILM, porque eles não avisam.

```
PUT teste_ilm-000001 -> { "acknowledged": true, "shards_acknowledged": true }
GET teste_ilm-000001/_ilm/explain -> step: ERROR
    step_info: { "type": "illegal_argument_exception",
                  "reason": "policy [politica_fantasma] does not exist" }
```

Lição: Aceitação da configuração não é validação da configuração. Em sistemas com componentes assíncronos, "acknowledged: true" significa apenas que a instrução foi registrada — não que ela vá funcionar.

PUT _template é a API legada do 7.x ELASTICSEARCH

SINTOMA	Todos os PUT _template do roteiro funcionam, mas cada um devolve um aviso amarelo de depreciação no Dev Tools.
CAUSA RAIZ	A API de templates foi reescrita no 7.8 e a antiga entrou em depreciação. A nova é PUT _index_template, com settings, mappings e aliases aninhados dentro de um objeto template, e com suporte a composable templates — blocos reutilizáveis combinados por prioridade.
DIAGNÓSTICO	Ler os avisos de depreciação em vez de ignorá-los. Eles são a única antecipação que se tem de uma quebra em versão futura.
CORREÇÃO	Migrar para PUT _index_template. A estrutura muda: o que estava na raiz passa a ficar dentro de "template": { "settings": {...}, "mappings": {...} }.

```
PUT _template/meu_template -> 200 OK + warning de depreciação
```

Lição: Aviso de depreciação é dívida com prazo. Funciona hoje, quebra numa atualização maior — e quase sempre no pior momento.

O BM25 conta documentos que já foram deletados ELASTICSEARCH

SINTOMA	O explain de uma consulta num índice com 2 documentos reporta N = 3, "total number of documents with field". O número não bate com a contagem do índice.
CAUSA RAIZ	Segmentos do Lucene são imutáveis. Um update não altera o documento: grava uma versão nova e marca a anterior como deletada. As estatísticas de frequência usadas pelo BM25 são lidas do segmento e incluem os documentos marcados, até que ocorra um merge.
DIAGNÓSTICO	Comparar docs.count com docs.deleted em _cat/indices e cruzar com o N do explain. No experimento controlado: 2 documentos, 1 update, docs.deleted = 1, e N passou de 2 para 3.
CORREÇÃO	Nenhuma correção é necessária — é o comportamento projetado. O efeito no ranking é desprezível em índices grandes e some no merge. O que se corrige é a expectativa de quem lê o explain.

```
2 docs vivos, 0 deletados -> n=2, N=2
apos 1 update             -> n=3, N=3 (docs.count=2, docs.deleted=1)
```

Lição: Score de relevância depende de estatísticas do corpus, e essas estatísticas são de segmento, não de uma contagem lógica. É por isso que o mesmo documento pode receber scores levemente diferentes em shards diferentes.

A Parte 7 do roteiro atualiza uma coleção que ainda não existe MONGODB

SINTOMA	Os comandos da Parte 7 rodam sem erro e reportam zero documentos modificados. Nada acontece, e nada avisa.
CAUSA RAIZ	O roteiro usa db.estudantes na Parte 7, mas a coleção só passa a se chamar assim depois do renameCollection da Parte 10. Como o MongoDB cria coleção de forma preguiçosa, o update contra um nome inexistente cria uma coleção vazia e atualiza nada.
DIAGNÓSTICO	Ler matchedCount e modifiedCount no retorno. Ambos em zero, sem erro, significa que o filtro não casou — ou que a coleção está vazia por não ser a que se pensava.
CORREÇÃO	Rodar a Parte 7 contra db.alunos, que é o nome da coleção naquele momento do roteiro.

```
db.estudantes.updateOne(...) -> { matchedCount: 0, modifiedCount: 0 }
```

Lição: Ausência de erro não é evidência de sucesso. Em sistemas que criam recursos sob demanda, errar o nome produz silêncio, não exceção — e silêncio é mais difícil de depurar que falha.

Os comentários do roteiro quebram no mongosh MONGODB

SINTOMA	Colar um bloco do enunciado no playground do VS Code produz SyntaxError, mesmo com os comandos corretos.
CAUSA RAIZ	O roteiro comenta com #, que é sintaxe de shell. O mongosh e o playground executam JavaScript, onde o comentário é //. E os atalhos do shell (use, show dbs, exit) também não existem no playground: lá é use("dbNoSQLBD") e db.getCollectionNames().
DIAGNÓSTICO	A mensagem de SyntaxError aponta a linha do comentário, não o comando — o que despista.
CORREÇÃO	Trocar # por // e usar a forma de função dos comandos de shell no playground.

```
show dbs -> SyntaxError no playground | db.getMongo().getDBNames() -> funciona
```

Lição: O mongosh parece um shell mas é um interpretador JavaScript. Saber qual linguagem está sendo interpretada evita uma classe inteira de erros que parecem inexplicáveis.

Anexo B — Exercícios

Não repetem o passo a passo dos laboratórios: exigem transferir o que foi aprendido para um problema novo. As soluções rodam nos ambientes dos labs.

1 Modelar sem repetir o modelo relacional

MONGODB

BÁSICO

15 MIN

Você precisa guardar pedidos de uma loja. Cada pedido tem cliente, data, endereço de entrega e uma lista de itens, cada item com produto, quantidade e preço unitário. Um colega propôs três coleções — pedidos, itens e endereços — ligadas por identificadores, como seria em SQL. Proponha a modelagem em documento e defenda a decisão de aninhar ou referenciar cada parte.

Espera-se: Um documento de exemplo e a justificativa de cada escolha, considerando como o dado é lido e como ele muda ao longo do tempo.

SOLUÇÃO

```
db.pedidos.insertOne({
  cliente_id: ObjectId("..."),           // REFERENCIA: cliente vive alem do pedido
  data: new Date("2026-09-09T10:00:00Z"),
  endereco_entrega: {                    // ANINHADO: e uma copia historica
    logradouro: "SQN 210 Bloco B",
    cidade: "Brasília", uf: "DF", cep: "70862-020"
  },
  itens: [                                // ANINHADO: nao existe fora do pedido
    { produto: "Teclado", quantidade: 1, preco_unitario: 250.00 },
    { produto: "Monitor", quantidade: 2, preco_unitario: 899.90 }
  ],
  total: 2049.80
});
```

A regra é perguntar o que é lido junto e o que muda junto. Os itens não existem fora do pedido e são sempre lidos com ele: aninhar evita junção e garante atomicidade, já que a escrita de um documento é atômica. O endereço é aninhado por outro motivo — ele precisa ser uma cópia congelada. Se fosse referência ao cadastro do cliente, mudar o endereço hoje reescreveria o histórico de entregas de dois anos atrás. Já o cliente é referenciado porque tem vida própria, é compartilhado por muitos pedidos e é atualizado independentemente. O total é redundante e calculável, mas evita recomputar em toda listagem — desnormalização deliberada.

Critério de correção: A resposta precisa justificar cada decisão pelo padrão de leitura e pelo ciclo de vida do dado. Aninhar tudo ou referenciar tudo, sem argumento, não atende. Identificar que o endereço é cópia histórica é o ponto de maior valor.

2 Um pipeline que responde a uma pergunta de negócio

MONGODB

INTERMEDIARIO

20 MIN

Usando a coleção de pedidos do exercício anterior, produza um relatório dos três produtos que mais geraram receita em 2026, com receita total e quantidade vendida, considerando apenas pedidos da cidade de Brasília.

Espera-se: Um pipeline de agregação com os estágios na ordem correta e a justificativa da posição do `$match`.

SOLUÇÃO

```
db.pedidos.aggregate([
  { $match: {
    "endereco_entrega.cidade": "Brasília",
    data: { $gte: new Date("2026-01-01"), $lt: new Date("2027-01-01") }
  } },
  { $unwind: "$itens" },
  { $group: {
    _id: "$itens.produto",
    receita: { $sum: { $multiply: ["$itens.quantidade", "$itens.preco_unitario"] } },
    unidades: { $sum: "$itens.quantidade" }
  } },
  { $sort: { receita: -1 } },
  { $limit: 3 }
]);
```

O `$match` vem primeiro por dois motivos: reduz o volume que entra no `$unwind`, que multiplica documentos, e é a única posição em que consegue usar índice. Um índice `{ "endereco_entrega.cidade": 1, data: 1 }` atenderia esse filtro. O `$unwind` é necessário porque `itens` é array e o `$group` precisa de um documento por item. Note que a receita é calculada dentro do `$sum` com `$multiply` — não existe campo receita no documento, ele é derivado no próprio estágio.

Critério de correção: O `$match` precisa estar antes do `$unwind`. Colocá-lo depois produz o mesmo resultado com custo maior e sem índice — vale correção parcial, com a justificativa cobrada.

3 Diagnosticar um índice que não está servindo

MONGODB

INTERMEDIARIO

15 MIN

Uma consulta `db.pedidos.find({ status: "entregue" }).sort({ data: -1 }).limit(20)` está lenta. Existe o índice `{ status: 1 }`. O `explain` mostra `IXSCAN`, mas também um estágio `SORT` e `totalDocsExamined` muito acima de 20. Explique o que está acontecendo e proponha a correção.

Espera-se: O diagnóstico do estágio `SORT` em memória e um índice composto que o elimine.

SOLUÇÃO

```
// diagnostico
db.pedidos.find({ status: "entregue" }).sort({ data: -1 }).limit(20)
  .explain("executionStats");
// stage: SORT  <- ordenacao em memoria, apos buscar todos os "entregue"

// correcao
db.pedidos.createIndex({ status: 1, data: -1 });

// depois: o SORT desaparece, totalDocsExamined cai para ~20
```

O índice `{ status: 1 }` serve ao filtro, mas não à ordenação. O servidor busca todos os pedidos entregues — que podem ser milhões — e só então ordena por data em memória, para no fim descartar tudo menos vinte. O índice composto `{ status: 1, data: -1 }` resolve os dois de uma vez: percorre apenas a faixa de status entregue, já na ordem de data decrescente, e para assim que junta vinte documentos. Se o `SORT` exceder o limite de memória, o MongoDB aborta a consulta — outro sintoma do mesmo problema.

Critério de correção: Identificar que o estágio `SORT` é o custo real, e que a ordem e a direção dos campos no índice composto importam. Propor apenas `{ data: -1 }` separado não resolve.

4 Escolher entre text e keyword

ELASTICSEARCH

BÁSICO

15 MIN

Um índice de artigos precisa: buscar por palavras no título, listar quantos artigos existem por autor, filtrar por tag exata e ordenar por data. Escreva o mapping e justifique o tipo de cada campo.

Espera-se: Um mapping com a decisão text/keyword/multi-field explicada campo a campo.

SOLUÇÃO

```
PUT artigos
{ "mappings": { "properties": {
  "titulo": { "type": "text",
    "fields": { "keyword": { "type": "keyword", "ignore_above": 256 } } },
  "autor": { "type": "text",
    "fields": { "keyword": { "type": "keyword", "ignore_above": 256 } } },
  "tags": { "type": "keyword" },
  "publicado_em": { "type": "date" }
} } }

// agregar por autor usa o sub-campo, nao o campo analisado:
GET artigos/_search
{ "size": 0, "aggs": { "por_autor": { "terms": { "field": "autor.keyword" } } } }
```

titulo é text porque precisa de busca full-text; ganha .keyword por precaução, caso venha a ser ordenado. autor precisa das duas capacidades ao mesmo tempo — buscar por nome parcial e agrupar por nome inteiro — e por isso o multi-field é obrigatório: agregar por autor analisado agruparia por primeiro nome e sobrenome separadamente. tags é keyword puro porque é vocabulário fechado e nunca será buscado por palavra solta. Note que um campo keyword aceita array sem declaração especial: no Elasticsearch todo campo é implicitamente multivalorado.

Critério de correção: O ponto central é perceber que autor exige multi-field. Declarar autor apenas como text inviabiliza a agregação; apenas como keyword inviabiliza a busca parcial.

5 must ou filter: onde cada cláusula pertence

ELASTICSEARCH

INTERMEDIÁRIO

15 MIN

Monte a consulta: artigos que falem sobre "bancos de dados distribuídos", publicados em 2026, com a tag nosql, ordenados por relevância. Decida o que vai em must e o que vai em filter, e justifique.

Espera-se: Uma bool query com a separação correta e a justificativa em termos de score e cache.

SOLUÇÃO

```
GET artigos/_search
{ "query": { "bool": {
  "must": [
    { "match": { "titulo": "bancos de dados distribuídos" } }
  ],
  "filter": [
    { "term": { "tags": "nosql" } },
    { "range": { "publicado_em": { "gte": "2026-01-01", "lt": "2027-01-01" } } }
  ]
} } }
```

Só o match vai em must, porque só ele exprime "quão bem este artigo responde à pergunta". Tag e intervalo de data são critérios binários: ou o documento satisfaz ou não, e nada neles torna um artigo mais relevante que outro. Em filter, o Elasticsearch pula o cálculo de score e pode reaproveitar o resultado em cache de bitset entre consultas — o mesmo filtro de ano será reutilizado por milhares de buscas diferentes. Colocar tudo em must funcionaria e devolveria os mesmos documentos, mas paga cálculo desnecessário e perde o cache.

Critério de correção: A separação precisa estar correta e a justificativa precisa mencionar score e cache. Justificar apenas por "boa prática" não atende.

6 Uma agregação que responde a uma pergunta editorial

ELASTICSEARCH

INTERMEDIARIO

15 MIN

Produza, em uma única consulta, os cinco autores com mais artigos publicados em 2026 e, para cada um, a data do artigo mais recente. Não traga nenhum documento no resultado.

Espera-se: Uma agregação terms com sub-agregação e size: 0.

SOLUÇÃO

```
GET artigos/_search
{
  "size": 0,
  "query": { "range": { "publicado_em": { "gte": "2026-01-01", "lt": "2027-01-01" } } },
  "aggs": {
    "top_autores": {
      "terms": { "field": "autor.keyword", "size": 5, "order": { "_count": "desc" } },
      "aggs": {
        "mais_recente": { "max": { "field": "publicado_em" } }
      }
    }
  }
}
```

Agregações aninham: a max roda dentro de cada bucket produzido pela terms. O size: 5 limita os buckets; o size: 0 no nível da consulta impede que documentos sejam devolvidos. Um detalhe importante para produção: em cluster com vários shards, o doc_count da terms é aproximado, porque cada shard devolve seu top parcial. O campo sum_other_doc_count na resposta informa quantos documentos ficaram fora dos buckets retornados.

Critério de correção: Precisa ter size: 0, usar o sub-campo keyword e aninhar a sub-agregação dentro da terms. Mencionar a aproximação em cluster multi-shard é diferencial.

7 Por que a busca semântica não achou o código do produto

VETORIAL

DESAFIO

20 MIN

Uma busca semântica sobre catálogo funciona bem para "notebook leve para viagem", mas falha quando o usuário digita o código exato "XPS-13-9370" — o produto existe e não é retornado. Explique a causa e proponha uma solução.

Espera-se: O diagnóstico do ponto cego da busca vetorial e uma estratégia híbrida.

SOLUÇÃO

```
GET catalogo/_search
{
  "query": {                                     // ramo lexical: pega o código exato
    "bool": { "should": [
      { "term": { "sku": "XPS-13-9370" } },
      { "match": { "descricao": "XPS-13-9370" } }
    ] }
  },
  "knn": {                                       // ramo semântico: pega a intenção
    "field": "embedding",
    "query_vector": [ /* embedding da consulta */ ],
    "k": 10, "num_candidates": 100
  },
  "rank": { "rrf": { "rank_window_size": 50, "rank_constant": 20 } }
}
```

Modelos de embedding são treinados em linguagem natural. Um código como XPS-13-9370 é tokenizado em fragmentos sem significado semântico, e o vetor resultante não se aproxima do vetor do produto — a busca vetorial é estruturalmente cega a identificadores. Já o BM25 acerta esse caso com facilidade, porque o termo é raro e o IDF o premia. A solução é híbrida, e o RRF é a forma correta de fundir: em vez de somar scores de escalas incompatíveis, ele soma o inverso da posição de cada documento em cada ranking, o que dispensa normalização. Um campo sku declarado como keyword garante a correspondência exata.

Critério de correção: Identificar que o ponto cego é estrutural, não um defeito de configuração, e propor combinação com busca lexical. Propor apenas "aumentar o k" não atende.

8 Onde mora a verdade

INTEGRADOR

DESAFIO

20 MIN

Um time propõe: "vamos gravar os pedidos direto no Elasticsearch, já que precisamos de busca e ele é mais rápido para consultar. Elimina o MongoDB e um ponto de sincronização." Você é responsável pela arquitetura. Responda ao time com argumentos técnicos e proponha uma alternativa.

Espera-se: Uma análise que identifique os riscos concretos e desenhe o fluxo alternativo.

SOLUÇÃO

```
// Fluxo proposto – a verdade em um lugar so, indice derivado e reconstruível

aplicacao
|
v escrita confirmada (write concern majority)
[ MongoDB ] <-- fonte da verdade
|
v change stream (ou CDC)
[ pipeline de sincronizacao ] <-- reexecutavel do zero
|
v bulk
[ Elasticsearch ] <-- indice derivado, descartavel

// reconstrucao completa deve ser um procedimento normal, nao uma emergencia:
// 1. cria indice novo com o mapping atualizado
// 2. reindexa a partir do MongoDB
// 3. troca o alias para o indice novo
// 4. remove o antigo
```

Os riscos concretos: o Elasticsearch é near real-time, então um pedido gravado não é imediatamente pesquisável, o que quebra o fluxo de confirmação ao cliente; não há transação multi-documento, então baixar estoque e criar pedido não podem ser atômicos; e mudar o tipo de um campo exige reindexar — o que, sem fonte externa, significa reindexar a partir do próprio índice, carregando qualquer defeito de mapping adiante. O argumento decisivo é esse último: um índice derivado pode ser reconstruído porque existe uma fonte; se ele for a fonte, toda evolução de esquema vira migração de risco. A alternativa mantém a busca — o time ganha o que queria — sem pôr a verdade num sistema projetado para ser descartável.

Critério de correção: Precisa citar pelo menos dois riscos técnicos concretos (near real-time, ausência de transação, reindexação) e desenhar um fluxo em que o índice seja reconstruível. Responder apenas "não é boa prática" não atende.

9 A busca que falha só para alguns usuários

INTEGRADOR

INTERMEDIÁRIO

15 MIN

Usuários relatam que buscar por "Sao Paulo" não retorna nada, enquanto "São Paulo" funciona. O campo é text com analyzer standard. Reproduza o diagnóstico e implemente a correção.

Espera-se: O diagnóstico via `_analyze` e um analyzer customizado com `asciifolding`.

SOLUÇÃO

```
// 1. diagnostico: os tokens preservam o acento
POST _analyze
{ "analyzer": "standard", "text": "São Paulo" }
// -> [ "são", "paulo" ] e "sao" nunca casa com "são"

// 2. correcao: analyzer com asciifolding, aplicado na indexacao e na busca
PUT cidades
{
  "settings": { "analysis": { "analyzer": {
    "pt_sem_acento": {
      "tokenizer": "standard",
      "filter": [ "lowercase", "asciifolding" ]
    }
  } } },
  "mappings": { "properties": {
    "nome": {
      "type": "text",
      "analyzer": "pt_sem_acento",
      "fields": { "keyword": { "type": "keyword", "ignore_above": 256 } }
    }
  } }
}

// 3. verificacao
POST cidades/_analyze
{ "field": "nome", "text": "São Paulo" } // -> [ "sao", "paulo" ]
```

A verificação com `_analyze` antes e depois é o que transforma suposição em diagnóstico. O `asciifolding` precisa valer para indexação e busca — se só a busca dobrasse o acento, "sao" continuaria não encontrando o termo "são" gravado no índice. Detalhe operacional importante: mudar analyzer de um campo existente exige reindexar, porque os termos já gravados foram produzidos pelo analyzer antigo. Este é exatamente o erro que o roteiro do Lab 2 comete ao afirmar que o standard já resolveria isso.

Critério de correção: Precisa usar `_analyze` como diagnóstico, aplicar `asciifolding` e reconhecer que a mudança exige reindexação dos dados existentes.

10 Estimar antes de indexar

ELASTICSEARCH

DESAFIO

20 MIN

Você vai indexar 5 milhões de documentos com embeddings de 768 dimensões para um sistema de RAG. Estime a memória necessária para os vetores, com e sem quantização int8, e discuta as implicações da escolha.

Espera-se: O cálculo numérico e uma discussão do compromisso entre memória, precisão e custo.

SOLUÇÃO

```
// float32: 4 bytes por dimensao
5.000.000 x 768 x 4 bytes = 15.360.000.000 bytes  ~= 14,3 GiB

// int8 (padrao do ES 8.x): 1 byte por dimensao
5.000.000 x 768 x 1 byte  = 3.840.000.000 bytes  ~= 3,6 GiB

// e ainda ha o grafo HNSW, proporcional a m (default 16)

// conferir o que o mapping realmente aplicou:
GET meu_indice/_mapping
// "index_options": { "type": "int8_hnsw", "m": 16, "ef_construction": 100 }
```

Quatro vezes menos memória é a diferença entre caber e não caber no heap de um nó — e o HNSW só é rápido quando o grafo está em memória. O custo é um erro de precisão que aparece na terceira ou quarta casa decimal do score, como se mediu no laboratório: o knn ficou levemente abaixo do valor teórico de $(1+\cos)/2$, justamente por causa da quantização. Em recuperação para RAG isso é irrelevante, porque o que importa é o conjunto dos k vizinhos, não o valor exato do score. Seria relevante se o score fosse usado como limiar absoluto de decisão. Vale notar que reduzir dimensões do modelo, quando ele suporta, costuma economizar mais que quantizar.

Critério de correção: O cálculo precisa estar correto e a discussão precisa reconhecer que a perda de precisão é aceitável para ranking e problemática para limiar absoluto. Lembrar do grafo HNSW além dos vetores é diferencial.

Anexo C — Avaliação

Questões de raciocínio, não de memorização de sintaxe. O gabarito explica por que cada alternativa errada é tentadora.

1. Um índice do Elasticsearch está com status yellow. O que se pode afirmar com certeza?

- A. Há perda de dados e o índice precisa ser restaurado de backup.
- B. Todos os shards primários estão ativos, mas pelo menos uma réplica não foi alocada.
- C. O cluster está sobrecarregado e rejeitando escritas.
- D. O mapping do índice está inconsistente com os documentos gravados.

Resposta: B — Yellow significa primários ativos e alguma réplica não alocada — o índice está plenamente funcional, apenas sem redundância. A alternativa A confunde yellow com red, que é o estado em que falta primário. C e D são tentadoras porque yellow parece alerta de saúde, mas o estado do cluster reporta apenas alocação de shards, nada sobre carga ou mapping.

2. `db.alunos.updateOne({ nome: "Ana" }, { idade: 24 })` foi executado. Qual o resultado?

- A. idade de Ana é atualizada para 24 e os demais campos permanecem.
- B. O comando falha com erro de sintaxe por faltar operador.
- C. O documento inteiro é substituído e passa a conter apenas idade: 24.
- D. Nada acontece, porque o segundo argumento não contém operador de atualização.

Resposta: C — Sem operador, o MongoDB interpreta o segundo argumento como substituição integral: nome e todos os outros campos são perdidos. A alternativa A é o que quase todo mundo espera, e é justamente por isso que o erro é perigoso. B é tentadora porque um erro seria o comportamento defensivo — mas não há erro. D descreveria um no-op, que também não ocorre.

3. Por que uma agregação `terms` sobre um campo declarado apenas como `text` falha?

- A. Porque campos `text` não são indexados.
- B. Porque o campo foi tokenizado e não existe um valor único por documento para formar buckets.
- C. Porque agregações só funcionam sobre campos numéricos.
- D. Porque falta declarar `doc_values` no mapping.

Resposta: B — O campo `text` é quebrado em termos pelo `analyzer`, então "São Paulo" vira dois termos e não há um valor íntegro para agrupar. A alternativa A é falsa — `text` é indexado, é justamente o que o torna pesquisável. C é falsa, pois `terms` agrega sobre keyword normalmente. D chega perto do mecanismo real (campos `text` não têm `doc_values` habilitados), mas a causa raiz é a tokenização, e habilitar `fielddata` seria a solução errada.

4. Você precisa que um sistema de e-commerce baixe estoque e registre a venda de forma atômica, e também ofereça busca por relevância no catálogo. Qual arranjo é adequado?

- A. Somente Elasticsearch, usando bulk para agrupar as duas operações.
- B. Somente MongoDB, implementando a busca com expressões regulares.
- C. MongoDB como fonte da verdade e Elasticsearch como índice derivado, sincronizados por um pipeline reexecutável.
- D. Elasticsearch como fonte da verdade e MongoDB como cache de leitura.

Resposta: C — Cada sistema no papel para o qual foi construído. A alternativa A falha porque bulk não é transação: agrupa requisições, não garante atomicidade. B funciona mal — regex não ordena por relevância e não usa índice invertido, degradando com o volume. D inverte os papéis e põe a verdade num sistema near real-time, sem transação e cuja evolução de esquema exige reindexação.

5. O explain de uma consulta MongoDB mostra IXSCAN, um estágio SORT e totalDocsExamined muito maior que nReturned. Qual a interpretação?

- A. O índice está corrompido e precisa ser reconstruído.
- B. O índice atende ao filtro mas não à ordenação, então o servidor ordena em memória.
- C. consulta não usa índice algum.
- D. O número alto de documentos examinados é normal quando há limit.

Resposta: B — A presença simultânea de IXSCAN e SORT é a assinatura desse problema: o índice serviu ao filtro, mas a ordenação teve de ser feita depois, em memória, sobre todos os documentos que casaram. C é contrariada pelo próprio IXSCAN. A é raro e não se diagnostica assim. D é a tentadora: com um índice adequado, o limit permite parar cedo justamente porque não há SORT no caminho.

6. Qual afirmação sobre o teorema CAP é correta?

- A. Todo sistema distribuído deve escolher permanentemente dois entre consistência, disponibilidade e tolerância a partição.
- B. Durante uma partição de rede, o sistema precisa escolher entre responder com dado possivelmente desatualizado ou recusar a resposta.
- C. Bancos NoSQL são AP e bancos relacionais são CA, por definição do modelo de dados.
- D. Tolerância a partição pode ser dispensada em sistemas com rede confiável.

Resposta: B — O teorema descreve a escolha durante a partição, não uma propriedade permanente. A alternativa A é a versão popular e distorcida — fora de partição, um sistema pode ser consistente e disponível ao mesmo tempo. C é falsa porque a classificação depende de configuração, não de produto ou modelo de dados. D é sedutora, mas partição não é escolha de projeto: é evento que ocorre em qualquer rede real.

7. Ao comparar script_score com cosineSimilarity e a query knn sobre os mesmos vetores, observou-se scores 1,99895 e 0,99936 para o mesmo documento. Por quê?

- A. query knn calculou errado, porque é aproximada.
- B. São fórmulas diferentes: script_score usa $\cos + 1,0$ e o knn usa $(1 + \cos) / 2$, e o knn ainda opera sobre vetores quantizados.
- C. O script_score considerou outros documentos no cálculo do score.
- D. O knn normalizou os vetores antes de comparar e o script_score não.

Resposta: B — As faixas são diferentes por definição — o a 2 contra o a 1 — e a pequena diferença restante vem da quantização int8 do HNSW. A alternativa A trata "aproximado" como "errado": o ranking foi idêntico nos dois caminhos. C descreve o BM25, cujo score depende de estatísticas do corpus, não o cosseno, que é calculado par a par. D inverte: a similaridade de cosseno já é invariante ao comprimento, por construção.

8. Um _bulk com mil documentos retornou HTTP 200. O que se pode concluir?

- A. Todos os mil documentos foram indexados com sucesso.
- B. requisição foi processada, mas pode haver falhas por item, que só o campo errors revela.
- C. Os documentos foram aceitos e já estão pesquisáveis.
- D. Nenhum documento falhou, mas alguns podem estar aguardando refresh.

Resposta: B — O bulk devolve 200 mesmo com falhas parciais — é obrigatório inspecionar errors e o status de cada item. A é o erro clássico. C soma dois enganos: além da falha parcial, ignora o near real-time. D é a mais sutil: acerta o refresh, mas afirma indevidamente que nenhum item falhou.

9. Um índice do Elasticsearch com 2 documentos reporta N = 3 no explain do BM25. Qual a explicação?

- A. Erro de contagem que se corrige com refresh.
- B. Há um documento em outro shard que não aparece na contagem.
- C. As estatísticas são lidas do segmento e incluem documentos marcados como deletados por um update anterior.
- D. O BM25 soma 1 ao total para evitar divisão por zero.

Resposta: C — Segmentos Lucene são imutáveis: um update grava versão nova e marca a antiga como deletada, e as estatísticas contam a marcada até o merge. A alternativa D é tentadora porque a fórmula do IDF realmente tem constantes de suavização — mas elas são 0,5, e não explicam N. A e B propõem causas plausíveis que o experimento controlado descarta.

10. Um índice foi criado a partir de um template que aponta para uma política ILM inexistente. O que acontece?

- A. criação do índice falha com erro explícito.
- B. O índice é criado normalmente e o ILM ignora o setting.
- C. O índice é criado com acknowledged: true, e o erro só aparece de forma assíncrona em `_ilm/explain`.
- D. O template é rejeitado no momento em que é registrado.

Resposta: C — A validação não ocorre na criação: o setting é apenas gravado, e quem tenta resolver a política é o serviço de ILM, em segundo plano. Verificado em teste: acknowledged: true na criação e step: ERROR no `_ilm/explain`. As alternativas A e D descrevem o comportamento defensivo que se esperaria; B parece inofensiva, mas o índice fica marcado como managed e nunca rotaciona.

TABELA DE DECISÃO

REQUISITO	SISTEMA	POR QUÊ
Transação envolvendo múltiplos documentos, com garantia de tudo-ou-nada	MongoDB	Transações multi-documento com ACID desde a versão 4.0. O Elasticsearch não oferece transação.
Busca por relevância em texto livre, ordenada por qualidade da correspondência	Elasticsearch	Índice invertido com BM25. No MongoDB, o índice de texto existe mas é limitado, e regex não usa índice.
Leitura do estado atual de um registro por identificador, imediatamente após a escrita	MongoDB	Leitura consistente logo após a confirmação da escrita. O Elasticsearch é near real-time e só torna o documento pesquisável no refresh seguinte.
Agregação exploratória sobre dezenas de milhões de eventos, com facetas e filtros combinados	Elasticsearch	Agregações distribuídas sobre estruturas colunares, executadas em paralelo por shard.
Busca semântica por significado, para RAG	Elasticsearch	dense_vector com HNSW e query knn nativa a partir da versão 8. O MongoDB Atlas oferece equivalente, mas não a edição community usada no laboratório.
Garantia de unicidade de um campo sob escrita concorrente	MongoDB	Índice único validado pelo servidor no momento da escrita, devolvendo E11000. O Elasticsearch só garante unicidade do _id.
Correlacionar logs de vários serviços numa linha do tempo comum	Elasticsearch	É o caso de uso para o qual a stack foi desenhada — foi exatamente o que o Lab 2 montou.
Fonte da verdade de qualquer dado de negócio	MongoDB	Índice de busca é derivado e descartável por princípio. Se ele for a fonte, toda evolução de mapping vira migração de risco, sem origem para reconstruir.

BSON

Formato binário do MongoDB que estende o JSON com tipos como data, decimal128, binário e ObjectId.

ObjectId

Identificador de 12 bytes gerado no cliente, contendo timestamp, valor aleatório por processo e contador.

Cursor

Ponteiro devolvido pelo find, que entrega documentos em lotes e aceita encadear sort, limit e skip.

COLLSCAN

Estágio do explain indicando que a coleção inteira foi varrida, documento por documento.

IXSCAN

Estágio em que o índice foi percorrido para localizar as chaves que casam com o filtro.

totalDocsExamined

Quantos documentos o servidor abriu para responder. Comparado a nReturned, mede a qualidade do índice.

Prefixo à esquerda

Regra segundo a qual um índice composto atende consultas sobre seus primeiros campos, na ordem declarada, mas não sobre os posteriores isoladamente.

Pipeline de agregação

Sequência de estágios em que cada um recebe a saída do anterior; a ordem altera o resultado e o custo.

\$facet

Estágio que executa vários pipelines independentes sobre a mesma entrada, numa única passada pelos dados.

Shard

Partição de um índice do Elasticsearch; cada shard é um índice Lucene completo e independente.

Réplica

Cópia de um shard primário, que oferece tolerância a falha e capacidade adicional de leitura.

Índice invertido

Estrutura que mapeia cada termo para a lista de documentos em que ele ocorre, com frequência e posição.

Analyzer

Cadeia que transforma texto em termos: filtros de caractere, tokenizador e filtros de token.

asciifolding

Filtro de token que converte caracteres acentuados em seus equivalentes ASCII, tornando a busca insensível a acento.

text e keyword

text é analisado e serve a busca full-text; keyword é um termo único e serve a filtro exato, ordenação e agregação.

BM25

Função de relevância padrão do Elasticsearch, combinando frequência do termo com saturação, frequência inversa nos documentos e normalização por tamanho.

Near real-time

Propriedade de o documento indexado só se tornar pesquisável após o refresh, que por padrão ocorre a cada segundo.

Alias

Nome estável que aponta para um ou mais índices físicos, permitindo trocá-los sem alterar a aplicação.

dense_vector

Tipo de campo que armazena um embedding, com número de dimensões fixo e igual ao do modelo que o gerou.

HNSW

Grafo navegável de mundo pequeno em camadas, que encontra vizinhos aproximados em tempo sublinear.

Similaridade de cosseno

Medida do ângulo entre dois vetores, de -1 a 1, invariante ao comprimento deles.

RRF

Reciprocal Rank Fusion: funde rankings somando o inverso da posição de cada documento, dispensando normalização de scores.

RAG

Recuperação de trechos relevantes numa base de conhecimento para compor o contexto de um modelo de linguagem antes da geração.

GELF

Graylog Extended Log Format; driver de log do Docker que envia a saída do contêiner por UDP a um coletor.

ILM

Index Lifecycle Management: automatiza rollover, migração entre camadas de armazenamento e expurgo de índices.

PARA IR ALÉM

- Documentação oficial do MongoDB, seção Aggregation Pipeline Optimization — explica quais reordenações o planejador faz sozinho e quais dependem de você.
- Elasticsearch: The Definitive Guide (Gormley e Tong) — anterior ao 8.x, mas a melhor explicação em português ou inglês sobre índice invertido, analyzers e relevância.
- Designing Data-Intensive Applications (Martin Kleppmann), capítulos 3 e 5 — estruturas de armazenamento e replicação, com o rigor que falta na maioria dos tutoriais.
- Artigo original do HNSW (Malkov e Yashunin, 2016) — leitura acessível para quem quer entender por que a busca aproximada funciona.
- Documentação do Elasticsearch sobre Retrievers e RRF — a forma atual de fazer busca híbrida, que substitui a combinação manual por `script_score` usada no laboratório.
- MongoDB University, curso M201 (Performance) — gratuito, e cobre explain e estratégia de índices com muito mais profundidade do que um laboratório permite.